Machine-checked certificate

Theorem 2 of G. Adesso, *Optimality of Gaussian entanglement of formation*
arXiv:2608.01909v3

Lean 4 (v4.35.0-rc4) + Mathlib (commit `49fd9de`) · github.com/gerardo-adesso/gaussian-eof @ `03f936b`

9 October 2026

Summary

All statements below are proved in Lean 4 against Mathlib, with no `sorry`, `admit` or additional axioms. The kernel reports that each depends only on `propext`, `Classical.choice` and `Quot.sound` (Sec. 7). They comprise Lemma S5 in full (positivity, kernel, and $M(x) \geq \gamma K(x)$ with the explicit constant $\gamma = \kappa \sinh w - w \coth w - |\alpha - \beta|/2$); the derivation of Eqs. (S:defects) and (S:Q12) from the Euler equation in the Schmidt-basis matrix model; the identity (S:mainid); the rigidity statement; and the scalar reduction from the minimum principle to Theorem 2. Statements are printed verbatim from the source; the full source is in the Appendix.

Not formalized (inputs)

- Operator facts on Fock space: the canonical commutation relations, which give the commutators $[T, O]$, the sum rules (S:wccr), and the matrix model with finitely many relevant directions; the TMSV moments and the identity $\lambda \hat{D}_t = T + \lambda$.
- The second-variation formula $Q(Y) = \mathrm{Re}\langle Y, (T - \mu - \mathsf{F})Y\rangle$ (Daleckii–Krein), the Euler equation (S:euler) and the stability inequality (S:stability).
- The vacuum/gap statement (S:gap): the joint vacuum of the canonical annihilators f_j, e_j is $\bigotimes_j \Psi_{q_j}$.
- The infinite-dimensional analysis: infinite Schmidt rank, Lemma S6, regularization, existence of minimizers.

These require operator theory on infinite-dimensional Hilbert spaces (unbounded ladder operators, trace-class operators, von Neumann entropy) not yet available in Mathlib. Comments in the printed source use plain letters in place of a few script glyphs ($\mathcal{E}$, $\mathcal{G}$, $\mathsf{A}, \mathsf{B}, \mathsf{C}, \mathsf{F}$).

1 Definitions

`f` — `Thm2Cert/MatrixLemma.lean`

```
/-- `f x = x / sinh x`, extended continuously by `f 0 = 1`. -/
def f (x : ℝ) : ℝ := if x = 0 then 1 else x / sinh x
```

`b` — `Thm2Cert/MatrixLemma.lean`

```
/-- `b x = x coth x`, written as `f x * cosh x` (so that `b 0 = 1`). -/
def b (x : ℝ) : ℝ := f x * cosh x
```

`Mdiag` — `Thm2Cert/MatrixLemma.lean`

```
/-- Diagonal entry of `M(x)` for a general `q` (`a = α` or `a = β`). -/
def Mdiag (κ a q x : ℝ) : ℝ := q * (κ - q * a) * exp x + (a - q * κ) * exp (-x) + (1 + q ^ 2) * f x
```

`Moff` — `Thm2Cert/MatrixLemma.lean`

```
/-- Off-diagonal entry of `M(x)` for a general `q`. -/
def Moff (κ q x : ℝ) : ℝ := -κ * (1 - q ^ 2) - 2 * q * b x
```

Hyp Thm2Cert/MatrixLemma.lean

```
/-- Hypotheses of Lemma S5: `α, β, κ > 0`, `αβ ≥ κ²`, and `w > 0` solves the calibration
`κ cosh w − w = (α+β)/2` (then `q = e^{−w}`). -/
structure Hyp (κ α β w : ℝ) : Prop where
  hκ : 0 < κ
  hα : 0 < α
  hβ : 0 < β
  hw : 0 < w
  hκαβ : κ ^ 2 ≤ α * β
  hcal : κ * cosh w - w = (α + β) / 2
```

QM Thm2Cert/MatrixLemma.lean

```
/-- The quadratic form of `M(x)`, `q = e^{−w}`. -/
def QM (κ α β w x u v : ℝ) : ℝ :=
  Mdiag κ α (exp (-w)) x * u ^ 2 + 2 * Moff κ (exp (-w)) x * u * v + Mdiag κ β (exp (-w)) x * v ^ 2
```

Kdiag Thm2Cert/Comparison.lean

```
/-- Diagonal entry of `K(x)`: `e^{−x} + q² eˣ` (the off-diagonal entry is `−2q`). -/
def Kdiag (q x : ℝ) : ℝ := exp (-x) + q ^ 2 * exp x
```

QK Thm2Cert/Comparison.lean

```
/-- The quadratic form of `K(x)`. -/
def QK (w x u v : ℝ) : ℝ :=
  Kdiag (exp (-w)) x * u ^ 2 + 2 * (-2 * exp (-w)) * u * v + Kdiag (exp (-w)) x * v ^ 2
```

gam Thm2Cert/Comparison.lean

```
/-- `γ = κ sinh w − w coth w − |α − β|/2`. -/
def gam (κ α β w : ℝ) : ℝ := κ * sinh w - w * cosh w / sinh w - |α - β| / 2
```

Lker Thm2Cert/Identity.lean

```
/-- The logarithmic mean kernel `L(p,p') = (ln p − ln p')/(p − p')`, `= 1/p` on the diagonal. -/
def Lker (p p' : ℝ) : ℝ := if p = p' then 1 / p else (log p - log p') / (p - p')
```

Fdk Thm2Cert/Derivation.lean

```
/-- `F(p, p') = (p ln p − p' ln p')/(p − p')`, `= 1 + ln p` on the diagonal. -/
def Fdk (p p' : ℝ) : ℝ := if p = p' then 1 + log p else (p * log p - p' * log p') / (p - p')
```

kk Thm2Cert/Identity.lean

```
/-- `k = Re(a b) = ∑ c_l c_r Re(A_lr B_lr)`. -/
def kk (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ := ∑ l, ∑ r, c l * c r * (A l r * B l r).re
```

sfA Thm2Cert/Identity.lean

```
/-- `A = ∑ p_l p_r L_lr |A_lr|²` (and `B` with `B`). -/
def sfA (c : ι → ℝ) (A : ι → ι → ℂ) : ℝ :=
  ∑ l, ∑ r, c l ^ 2 * c r ^ 2 * Lker (c l ^ 2) (c r ^ 2) * Complex.normSq (A l r)
```

sfC Thm2Cert/Identity.lean

```
/-- `C = ∑ √(p_l p_r) (p_l + p_r) L_lr Re(A_lr B_lr)`. -/
def sfC (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ :=
  ∑ l, ∑ r, √(c l ^ 2 * c r ^ 2) * (c l ^ 2 + c r ^ 2) * Lker (c l ^ 2) (c r ^ 2) * (A l r * B l r).re
```

Q1 Thm2Cert/Identity.lean

```
/-- Eq. (S:Q12), first line: `Q(O₁ψ)`. -/
def Q1 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q * κ - α) * nA + (q ^ 2 * β - q * κ) * (nB + 1) + (κ * (1 - q ^ 2) + q * (α - β)) * k
    - sA - q ^ 2 * sB + q * sC
```

Q2 Thm2Cert/Identity.lean

```
/-- Eq. (S:Q12), second line: `Q(O₂ψ)`. -/
def Q2 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q ^ 2 * α - q * κ) * (nA + 1) + (q * κ - β) * nB + (κ * (1 - q ^ 2) - q * (α - β)) * k
    - q ^ 2 * sA - sB + q * sC
```

O1v Thm2Cert/Derivation.lean

```
/-- Coefficient matrix of `O₁ψ = (a − q b†)ψ`. -/
def O1v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  A l m * (c m : ℂ) - (q : ℂ) * (c l : ℂ) * conj (B l m)
```

O2v Thm2Cert/Derivation.lean

```
/-- Coefficient matrix of `O₂ψ = (q a† − b)ψ`. -/
def O2v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  (q : ℂ) * (c m : ℂ) * conj (A m l) - (c l : ℂ) * B m l
```

kterm Thm2Cert/Comparison.lean

```
/-- Summand of `‖O₁ψ‖² + ‖O₂ψ‖² = ∑ c_l c_r z† K(x_lr) z` (Eq. S:defects) in the
`p`-variables: `c_l c_r K₁₁ = p_r + q² p_l`, `c_l c_r K₁₂ = −2q c_l c_r`. -/
def kterm (q cl cr : ℝ) (A B : ℂ) : ℝ :=
  (cr ^ 2 + q ^ 2 * cl ^ 2) * (Complex.normSq A + Complex.normSq B)
    - 4 * q * (cl * cr) * (A * B).re
```

term Thm2Cert/Identity.lean

```
/-- The summand of Eq. (S:mainid), for Schmidt coefficients `cl, cr ≥ 0`. -/
def term (κ α β q cl cr : ℝ) (A B : ℂ) : ℝ :=
  wdiag κ α q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq A
  + wdiag κ β q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq B
  + 2 * woff κ q (cl * cr) (√(cl ^ 2 * cr ^ 2) * (cl ^ 2 + cr ^ 2) * Lker (cl ^ 2) (cr ^ 2))
      * (A * B).re
```

lam Thm2Cert/Reduction.lean

```
/-- `λ_{q,t} = −2 q ln q / [(t − q)(1 − t q)]`. -/
def lam (q t : ℝ) : ℝ := -2 * q * log q / ((t - q) * (1 - t * q))
```

dt Thm2Cert/Reduction.lean

```
/-- `d_t(q) = (1 − t q)² / (1 − q²)`. -/
def dt (t q : ℝ) : ℝ := (1 - t * q) ^ 2 / (1 - q ^ 2)
```

gfun Thm2Cert/Reduction.lean

```
/-- `g(y) = (y+1) ln(y+1) − y ln y`. -/
def gfun (y : ℝ) : ℝ := (y + 1) * log (y + 1) - y * log y
```

EE Thm2Cert/Reduction.lean

```
/-- `E(q) = g(q²/(1−q²))`, the entanglement of the TMSV. -/
def EE (q : ℝ) : ℝ := gfun (q ^ 2 / (1 - q ^ 2))
```

2 Lemma S5: positivity and kernel

hyp_nonvacuous Thm2Cert/MatrixLemma.lean

```
/-- Sanity check: the hypotheses are satisfiable (`w = 1`, `κ = 2/(cosh 1 − 1)`,
`α = β = κ cosh 1 − 1`). -/
theorem hyp_nonvacuous : ∃ κ α β w : ℝ, Hyp κ α β w
```

lemmaS5_psd Thm2Cert/MatrixLemma.lean

```
/-- **Lemma S5 (i)**: `M(x) ≥ 0` for every real `x`. -/
theorem lemmaS5_psd (h : Hyp κ α β w) (x u v : ℝ) : 0 ≤ QM κ α β w x u v
```

lemmaS5_pd Thm2Cert/MatrixLemma.lean

```
/-- **Lemma S5 (ii)**: `M(x)` is positive definite for `x ≠ w`, i.e. `ker M(x) = {0}`. -/
theorem lemmaS5_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {u v : ℝ} (huv : u ≠ 0 ∨ v ≠ 0) :
    0 < QM κ α β w x u v
```

lemmaS5_kernel Thm2Cert/MatrixLemma.lean

```
/-- **Lemma S5 (iii)**: at `x = w`, the quadratic form vanishes exactly on `span{(1,1)}`,
i.e. `ker M(w) = span{(1,1)}`. -/
theorem lemmaS5_kernel (h : Hyp κ α β w) (u v : ℝ) : QM κ α β w w u v = 0 ↔ u = v
```

lemmaS5_null Thm2Cert/MatrixLemma.lean

```
/-- `M(w) (1,1)ᵀ = 0`, entrywise. -/
theorem lemmaS5_null :
    Mdiag κ α (exp (-w)) w + Moff κ (exp (-w)) w = 0 ∧
    Moff κ (exp (-w)) w + Mdiag κ β (exp (-w)) w = 0
```

lemmaS5_complex Thm2Cert/MatrixLemma.lean

```
/-- Complex form used in Eq. (S:mainid): for `z = (A, conj B)`,
`z† M(x) z = M₁₁|A|² + M₂₂|B|² + 2 M₁₂ Re(AB) ≥ 0`. -/
theorem lemmaS5_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    0 ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re
```

lemmaS5_complex_pd Thm2Cert/MatrixLemma.lean

```
/-- Complex strict form: for `x ≠ w` and `z = (A, conj B) ≠ 0`, `z† M(x) z > 0`. -/
theorem lemmaS5_complex_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {A B : ℂ}
    (hAB : A ≠ 0 ∨ B ≠ 0) :
    0 < Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re
```

lemmaS5_complex_kernel Thm2Cert/MatrixLemma.lean

```
/-- Complex kernel at `x = w`: `z† M(w) z = 0` iff `z = (A, conj B)` lies in `span{(1,1)}`. -/
theorem lemmaS5_complex_kernel (h : Hyp κ α β w) (A B : ℂ) :
    Mdiag κ α (exp (-w)) w * Complex.normSq A + Mdiag κ β (exp (-w)) w * Complex.normSq B
      + 2 * Moff κ (exp (-w)) w * (A * B).re = 0 ↔ A = (starRingEnd ℂ) B
```

3 Lemma S5: comparison with explicit constant

gam_pos Thm2Cert/Comparison.lean

```
theorem gam_pos (h : Hyp κ α β w) : 0 < gam κ α β w
```

lemmaS5_gamma Thm2Cert/Comparison.lean

```
/-- **Lemma S5 (iv)**: `M(x) ≥ γ K(x)` for every real `x`, with the explicit
`γ = κ sinh w − w coth w − |α−β|/2 > 0` (`gam_pos`). -/
theorem lemmaS5_gamma (h : Hyp κ α β w) (x u v : ℝ) :
    gam κ α β w * QK w x u v ≤ QM κ α β w x u v
```

lemmaS5_gamma_complex Thm2Cert/Comparison.lean

```
/-- Complex form of `M(x) ≥ γ K(x)`, for `z = (A, conj B)`. -/
theorem lemmaS5_gamma_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    gam κ α β w * (Kdiag (exp (-w)) x * (Complex.normSq A + Complex.normSq B)
        + 2 * (-2 * exp (-w)) * (A * B).re)
      ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
        + 2 * Moff κ (exp (-w)) x * (A * B).re
```

4 Eqs. (S:defects), (S:Q12), (S:mainid)

defects Thm2Cert/Derivation.lean

```
/-- **Eq. (S:defects)**: `‖O₁ψ‖² + ‖O₂ψ‖² = ∑_{l,r} c_l c_r z† K(x_lr) z` (in `p`-variables). -/
theorem defects (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) :
    ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) + ∑ l, ∑ m, Complex.normSq (O2v q c A B l m)
      = ∑ l, ∑ r, kterm q (c l) (c r) (A l r) (B l r)
```

Q1_from_euler Thm2Cert/Derivation.lean

```
/-- **Eq. (S:Q12), first line**, derived from the Euler equation and `[T, O₁]`. -/
theorem Q1_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
    (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    (∑ l, ∑ m, conj (O1v q c A B l m)
      * ((A l m * ((c m * log (c m ^ 2) : ℝ) : ℂ)
          - (q : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B l m)
          + ((q * κ - α : ℝ) : ℂ) * A l m * (c m : ℂ)
          + ((κ - q * β : ℝ) : ℂ) * (c l : ℂ) * conj (B l m))
        - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O1v q c A B l m)).re
    = Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
```

Q2_from_euler Thm2Cert/Derivation.lean

```
/-- **Eq. (S:Q12), second line**, derived from the Euler equation and `[T, O₂]`. -/
theorem Q2_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
    (α β κ q nA nB : ℝ)
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r)) :
    (∑ l, ∑ m, conj (O2v q c A B l m)
      * (((q : ℂ) * ((c m * log (c m ^ 2) : ℝ) : ℂ) * conj (A m l)
          - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B m l
          + ((q * α - κ : ℝ) : ℂ) * (c m : ℂ) * conj (A m l)
          + ((β - q * κ : ℝ) : ℂ) * (c l : ℂ) * B m l)
        - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O2v q c A B l m)).re
    = Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
```

mainid Thm2Cert/Identity.lean

```
/-- **Eq. (S:mainid)** (exact bookkeeping identity), for an arbitrary real `q`. -/
theorem mainid (c : ι → ℝ) (hc : ∀ l, 0 < c l) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
    = -∑ l, ∑ r, c l * c r *
        (Mdiag κ α q (log (c l / c r)) * Complex.normSq (A l r)
          + Mdiag κ β q (log (c l / c r)) * Complex.normSq (B l r)
          + 2 * Moff κ q (log (c l / c r)) * (A l r * B l r).re)
```

mainid_general Thm2Cert/Identity.lean

```
/-- **Eq. (S:mainid), general form**: valid for all real `c_l` (no positivity needed). -/
theorem mainid_general (c : ι → ℝ) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
    = -∑ l, ∑ r, term κ α β q (c l) (c r) (A l r) (B l r)
```

5 Rigidity

rigidity Thm2Cert/Comparison.lean

```
/-- **Rigidity inequality** for one pair: `Q(O₁ψ) + Q(O₂ψ) ≤ −γ ∑ c_l c_r z† K z`. -/
theorem rigidity (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l)
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      ≤ -(gam κ α β w * ∑ l, ∑ r, kterm (exp (-w)) (c l) (c r) (A l r) (B l r))
```

rigidity_multi Thm2Cert/Comparison.lean

```
/-- **Rigidity, many pairs.** For `m` pairs `j` (all sharing the Schmidt coefficients `c`)
with hypotheses `Hyp κ_j α_j β_j w_j`, and any `0 < γ₀ ≤ min_j γ_j`:
`∑_j [Q(O₁,ⱼψ) + Q(O₂,ⱼψ)] ≤ −γ₀ ∑_j ∑ c_l c_r z†_j K_j z_j`. In particular, if the left side
is `≥ 0` (second-order stability with `(O_{i,j}) = 0`), every defect sum vanishes. -/
theorem rigidity_multi {m : ℕ} (κ α β w : Fin m → ℝ) (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
    (γ0 : ℝ) (hγ0 : 0 < γ0) (hγ : ∀ j, γ0 ≤ gam (κ j) (α j) (β j) (w j))
    (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
    (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
    (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
    (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
    (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r)) :
    (∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))))
      ≤ -(γ0 * ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r)) ∧
    (0 ≤ ∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
```

```
        (sfA c (B j)) (sfC c (A j) (B j))) →
      ∀ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) = 0)
```

stationary_rigidity Thm2Cert/Derivation.lean

```
/-- **Rigidity.** For `m` pairs with the hypotheses of Lemma S5, Schmidt coefficients `c ≥ 0`
and the sum rules, if the second variations (S:Q12) along all test vectors `O₁,ⱼψ, O₂,ⱼψ` −
computed from the Euler equation − sum to a nonnegative number, then every test vector
vanishes: `(a_j − q_j b_j†)ψ = 0` and `(q_j a_j† − b_j)ψ = 0` for all `j`. -/
theorem stationary_rigidity {m : ℕ} (κ α β w : Fin m → ℝ)
    (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
    (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
    (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
    (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
    (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
    (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r))
    (hstab : 0 ≤ ∑ j,
      ((∑ l, ∑ m', conj (O1v (exp (-w j)) c (A j) (B j) l m')
        * ((A j l m' * ((c m' * log (c m' ^ 2) : ℝ) : ℂ)
            - ((exp (-w j) : ℝ) : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B j l m')
            + ((exp (-w j) * κ j - α j : ℝ) : ℂ) * A j l m' * (c m' : ℂ)
            + ((κ j - exp (-w j) * β j : ℝ) : ℂ) * (c l : ℂ) * conj (B j l m'))
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O1v (exp (-w j)) c (A j) (B j) l m')).re
      + (∑ l, ∑ m', conj (O2v (exp (-w j)) c (A j) (B j) l m')
        * ((((exp (-w j) : ℝ) : ℂ) * ((c m' * log (c m' ^ 2) : ℝ) : ℂ) * conj (A j m' l)
            - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B j m' l
            + ((exp (-w j) * α j - κ j : ℝ) : ℂ) * (c m' : ℂ) * conj (A j m' l)
            + ((β j - exp (-w j) * κ j : ℝ) : ℂ) * (c l : ℂ) * B j m' l)
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O2v (exp (-w j)) c (A j) (B j) l m')).re)) :
    ∀ j l m', O1v (exp (-w j)) c (A j) (B j) l m' = 0 ∧ O2v (exp (-w j)) c (A j) (B j) l m' = 0
```

6 Reduction to Theorem 2

calib_iff Thm2Cert/Reduction.lean

```
/-- The calibration for `(α, β, κ) = λ(t², 1, t)` holds iff `λ = λ_{q,t}`, `q = e^{−w}`. -/
theorem calib_iff {w t l : ℝ} (hw : 0 < w) (hqt : exp (-w) < t) (ht : t ≤ 1) :
    (l * t) * cosh w - w = (l * t ^ 2 + l * 1) / 2 ↔ l = lam (exp (-w)) t
```

lam_pos Thm2Cert/Reduction.lean

```
lemma lam_pos {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) : 0 < lam q t
```

hyp_of_lam Thm2Cert/Reduction.lean

```
/-- The parameters of Theorem 2 satisfy the hypotheses of Lemma S5: for `0 < q < t ≤ 1`,
`w = −ln q`, `λ = λ_{q,t}`, the triple `(α, β, κ) = λ(t², 1, t)` (and the swapped one
`λ(1, t², t)`) satisfies `Hyp`, with `κ² = αβ`. -/
theorem hyp_of_lam {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    Hyp (lam q t * t) (lam q t * t ^ 2) (lam q t * 1) (-log q) ∧
    Hyp (lam q t * t) (lam q t * 1) (lam q t * t ^ 2) (-log q)
```

tmsv_value Thm2Cert/Reduction.lean

```
/-- Equality in the minimum principle on the two-mode squeezed vacuum (scalar part). -/
theorem tmsv_value {κ α β w : ℝ} (hcal : κ * cosh w - w = (α + β) / 2) (hw : 0 < w) :
    EE (exp (-w)) + (α + β) * (exp (-w) ^ 2 / (1 - exp (-w) ^ 2))
      - κ * (2 * exp (-w) / (1 - exp (-w) ^ 2))
      = -log (1 - exp (-w) ^ 2) - κ * exp (-w)
```

energy_identity Thm2Cert/Reduction.lean

```
/-- `E(q) + λ_{q,t} d_t(q) = −ln(1−q²) − λ_{q,t} t q + λ_{q,t}`. -/
theorem energy_identity {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    EE q + lam q t * dt t q = -log (1 - q ^ 2) - lam q t * t * q + lam q t
```

thm2_of_minprinciple Thm2Cert/Reduction.lean

```
/-- **Theorem 2 from the minimum principle.** For `m` pairs with `0 < q_j < t_j ≤ 1`,
`λ_j = λ_{q_j,t_j}`, `κ_j = λ_j t_j`, and `⟨T_j⟩ = λ_j(⟨D̂^{(j)}⟩ − 1)`, the minimum principle
`S + ∑ ⟨T_j⟩ ≥ ∑ [−ln(1−q_j²) − κ_j q_j]` implies
`S + ∑ λ_j [⟨D̂^{(j)}⟩ − d_{t_j}(q_j)] ≥ ∑ E(q_j)`. -/
theorem thm2_of_minprinciple {m : ℕ} (q t D : Fin m → ℝ) (Sent : ℝ)
    (hq : ∀ j, 0 < q j) (hqt : ∀ j, q j < t j) (ht : ∀ j, t j ≤ 1)
    (hmin : Sent + ∑ j, lam (q j) (t j) * (D j - 1)
      ≥ ∑ j, (-log (1 - q j ^ 2) - (lam (q j) (t j) * t j) * q j)) :
    Sent + ∑ j, lam (q j) (t j) * (D j - dt (t j) (q j)) ≥ ∑ j, EE (q j)
```

7 Verification record

```
Lean toolchain: leanprover/lean4:v4.35.0-rc4
Mathlib commit: 49fd9dee36e7bbf345fe7981d2de6aeeb591f75b (8 Oct 2026)

== Clean build (lake build) ==
✔ [0/163] Ran Thm2Cert:extraDep
⚠ [2229/2235] Built Thm2Cert.MatrixLemma (9.6s)
⚠ [2230/2235] Built Thm2Cert.Reduction (4.6s)
⚠ [2231/2235] Built Thm2Cert.Identity (4.8s)
i [2232/2235] Built Thm2Cert.Comparison (12s)
i [2233/2235] Built Thm2Cert.Derivation (3.1s)
i [2234/2235] Built Thm2Cert (1.6s)
✔ [2235/2235] Ran Thm2Cert/Thm2Cert:default
Build completed successfully (2235 jobs).
real        0m33.205s
(only remaining warnings: Mathlib's generic 'consider adding module' notices)

== Axiom audit (lake env lean Certificate.lean) ==
'Thm2Cert.lemmaS5_psd' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_pd' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_kernel' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_null' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_complex' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_complex_pd' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_complex_kernel' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.hyp_nonvacuous' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.gam_pos' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_gamma' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lemmaS5_gamma_complex' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.defects' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.Q1_from_euler' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.Q2_from_euler' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.mainid' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.mainid_general' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.rigidity' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.rigidity_multi' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.stationary_rigidity' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.calib_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.lam_pos' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.hyp_of_lam' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.tmsv_value' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.energy_identity' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm2Cert.thm2_of_minprinciple' depends on axioms: [propext, Classical.choice, Quot.sound]

No sorry / admit / axiom declarations in Thm2Cert/*.lean (1836 lines).

== SHA-256 ==
86a3d914234d083870e9033a3f2a6502821ce7614638457b6cf3baf2f4914911  Thm2Cert.lean
b2d3d45a295b136a60b17340ca964628a5791ba9a6bc7c5ff2e0effca727f7cf  Certificate.lean
17cddd2c66982fdc705e5593c2f9d10719f5b1559ecbad9684e8cc5d64bde661  Thm2Cert/Comparison.lean
8c08ff7ada7a1b6188643d207fd00c94e4643c5b81f8d0910e2e9c07d71ede7f  Thm2Cert/Derivation.lean
dfdfc8f1d40ba675859a8e2a8ebc9507f366dd91ca778a47c7fc986b4bb9e520  Thm2Cert/Identity.lean
d444f89ef18c12792308cf0e2d40fcb15fca44b22992042c76c5d156453f62a9  Thm2Cert/MatrixLemma.lean
74cb74257d090f4f2c87d2ecaa22a74e24589ce711643345a28d13713e0978ef  Thm2Cert/Reduction.lean
89ad06bc4390342de18218fee4be2439961d29af84a105d1c7158401e2985bf6  lakefile.toml
8f89aa44fccdb1a0b6cc768c7c9f54b94cd40e47a08d582f82d1235f13075cf7  lean-toolchain
```

Appendix: full source

Thm2Cert/MatrixLemma.lean

```
/-
Copyright (c) 2026. Machine-checked certificate for
G. Adesso, "Optimality of Gaussian entanglement of formation", Supplemental Material,
Lemma S5 (the matrix inequality behind Theorem 2).

Statement certified here (all real numbers):
  Let κ, α, β, w > 0 with κ² ≤ αβ and the calibration κ cosh w − w = (α+β)/2,
  and q = e^{−w}.  With f(x) = x / sinh x (f(0) = 1) and b(x) = x coth x = f(x) cosh x, put
    M₁₁(x) = q(κ − qα)eˣ + (α − qκ)e⁻ˣ + (1+q²) f(x)
    M₂₂(x) = q(κ − qβ)eˣ + (β − qκ)e⁻ˣ + (1+q²) f(x)
    M₁₂(x) = −κ(1 − q²) − 2q b(x).
  Then M(x) is positive semidefinite for every real x, positive definite for x ≠ w,
  and the kernel of M(w) is spanned by (1,1).
The comparison M ≥ γK of Lemma S5 (with explicit γ) is certified in `Comparison.lean`.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.DerivHyp
import Mathlib.Analysis.Convex.Deriv
import Mathlib.Analysis.Complex.Norm

open Real

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-! ## The auxiliary functions `f x = x / sinh x` and `b x = x coth x` -/

/-- `f x = x / sinh x`, extended continuously by `f 0 = 1`. -/
def f (x : ℝ) : ℝ := if x = 0 then 1 else x / sinh x

/-- `b x = x coth x`, written as `f x * cosh x` (so that `b 0 = 1`). -/
def b (x : ℝ) : ℝ := f x * cosh x

@[simp] lemma f_zero : f 0 = 1 := by simp [f]

lemma f_mul_sinh (x : ℝ) : f x * sinh x = x := by
  unfold f
  split_ifs with h
  · simp [h]
  · have : sinh x ≠ 0 := sinh_ne_zero.mpr h
    field_simp

lemma f_pos (x : ℝ) : 0 < f x := by
  unfold f
  split_ifs with h
  · exact one_pos
  · rcases lt_or_gt_of_ne h with hx | hx
    · exact div_pos_of_neg_of_neg hx (sinh_neg_iff.mpr hx)
    · exact div_pos hx (sinh_pos_iff.mpr hx)

lemma f_neg (x : ℝ) : f (-x) = f x := by
  unfold f
  by_cases h : x = 0
  · simp [h]
  · have h' : -x ≠ 0 := neg_ne_zero.mpr h
    simp only [h, h', ite_false, sinh_neg, neg_div_neg_eq]

lemma b_eq {x : ℝ} (hx : x ≠ 0) : b x = x * cosh x / sinh x := by
  unfold b f; simp only [hx, ite_false]; ring

/-! ## Two monotonicity facts -/

lemma sinh_strictConvexOn : StrictConvexOn ℝ (Set.Ici (0:ℝ)) sinh := by
  apply StrictMonoOn.strictConvexOn_of_deriv (convex_Ici 0) continuous_sinh.continuousOn
  rw [interior_Ici, Real.deriv_sinh]
  exact cosh_strictMonoOn.mono Set.Ioi_subset_Ici_self

/-- (L1) `sinh t / t` is strictly increasing on `(0, ∞)`, in cross-multiplied form. -/
lemma L1 {a c : ℝ} (ha : 0 < a) (hac : a < c) : c * sinh a < a * sinh c := by
  have hc : 0 < c := ha.trans hac
  have h1 : (0:ℝ) < 1 - a / c := by rw [sub_pos, div_lt_one hc]; exact hac
  have h2 : (0:ℝ) < a / c := div_pos ha hc
  have h := sinh_strictConvexOn.2 (Set.self_mem_Ici) (Set.mem_Ici.mpr hc.le) hc.ne h1 h2
    (by ring)
  simp only [smul_eq_mul, mul_zero, zero_add, sinh_zero] at h
  rw [div_mul_cancel₀ a hc.ne'] at h
  calc c * sinh a < c * (a / c * sinh c) := mul_lt_mul_of_pos_left h hc
    _ = a * sinh c := by field_simp

/-- (L2) the identity behind the monotonicity of `sinh a / sinh (a + c)`. -/
lemma L2 (a a' c : ℝ) :
    sinh a' * sinh (a + c) - sinh a * sinh (a' + c) = sinh (a' - a) * sinh c := by
  simp only [sinh_add, sinh_sub]; ring
```

```
86
87 /-! ## Sign of `g(x) = w sinh x − sinh² w · f x` -/
88
89 lemma g_pos {w x : ℝ} (hw : 0 < w) (hx : w < x) : 0 < w * sinh x - sinh w ^ 2 * f x := by
90   have hx0 : 0 < x := hw.trans hx
91   have hs : 0 < sinh w := sinh_pos_iff.mpr hw
92   have hsx : sinh w < sinh x := sinh_lt_sinh.mpr hx
93   have hsxp : 0 < sinh x := hs.trans hsx
94   have hL := L1 hw hx
95   have key : (w * sinh x - sinh w ^ 2 * f x) * sinh x
96       = w * sinh x * sinh x - x * sinh w * sinh w := by
97     linear_combination (-(sinh w) ^ 2) * f_mul_sinh x
98   have h1 := mul_lt_mul_of_pos_right hL hsxp
99   have h2 := mul_lt_mul_of_pos_left hsx (mul_pos hx0 hs)
100  have h3 : 0 < (w * sinh x - sinh w ^ 2 * f x) * sinh x := by rw [key]; linarith
101  nlinarith
102
103 lemma g_neg {w x : ℝ} (hw : 0 < w) (hx : x < w) : w * sinh x - sinh w ^ 2 * f x < 0 := by
104   have hs : 0 < sinh w := sinh_pos_iff.mpr hw
105   have hfs : 0 < sinh w ^ 2 * f x := mul_pos (pow_pos hs 2) (f_pos x)
106   rcases le_or_gt x 0 with hx0 | hx0
107   · have : sinh x ≤ 0 := sinh_nonpos_iff.mpr hx0
108     nlinarith
109   · have hsx : sinh x < sinh w := sinh_lt_sinh.mpr hx
110     have hsxp : 0 < sinh x := sinh_pos_iff.mpr hx0
111     have hL := L1 hx0 hx
112     have key : (w * sinh x - sinh w ^ 2 * f x) * sinh x
113         = w * sinh x * sinh x - x * sinh w * sinh w := by
114       linear_combination (-(sinh w) ^ 2) * f_mul_sinh x
115     have h1 := mul_lt_mul_of_pos_right hL hsxp
116     have h2 := mul_lt_mul_of_pos_left hsx (mul_pos hx0 hs)
117     have h3 : (w * sinh x - sinh w ^ 2 * f x) * sinh x < 0 := by rw [key]; linarith
118     nlinarith
119
120 /-! ## Sign of `J(x) = w sinh(x/2) − sinh(w/2) f(x) sinh((x+w)/2)` (that is, `w · I(x)`) -/
121
122 /-- `J w x = w * I(x)` with `I` as in the proof of Lemma S5. -/
123 def J (w x : ℝ) : ℝ := w * sinh (x / 2) - sinh (w / 2) * f x * sinh ((x + w) / 2)
124
125 lemma J_key (w x : ℝ) : J w x * sinh x * sinh w
126     = (w * sinh x) * (sinh (x / 2) * sinh w)
127       - (x * sinh w) * (sinh (w / 2) * sinh ((x + w) / 2)) := by
128   unfold J
129   linear_combination (-(sinh (w / 2)) * sinh ((x + w) / 2) * sinh w) * f_mul_sinh x
130
131 lemma J_pos {w x : ℝ} (hw : 0 < w) (hx : w < x) : 0 < J w x := by
132   have hx0 : 0 < x := hw.trans hx
133   have hS : 0 < sinh w := sinh_pos_iff.mpr hw
134   have hs : 0 < sinh (w / 2) := sinh_pos_iff.mpr (by linarith)
135   have hsx : 0 < sinh x := sinh_pos_iff.mpr hx0
136   have hL := L1 hw hx
137   have h2 := L2 (w / 2) (x / 2) (w / 2)
138   rw [show w / 2 + w / 2 = w by ring, show x / 2 + w / 2 = (x + w) / 2 by ring] at h2
139   have h2' : sinh (w / 2) * sinh ((x + w) / 2) < sinh (x / 2) * sinh w := by
140     have : 0 < sinh (x / 2 - w / 2) * sinh (w / 2) :=
141       mul_pos (sinh_pos_iff.mpr (by linarith)) hs
142     linarith
143   have hpos : 0 < sinh (w / 2) * sinh ((x + w) / 2) :=
144     mul_pos hs (sinh_pos_iff.mpr (by linarith))
145   have hprod := mul_lt_mul'' hL h2' (mul_pos hx0 hS).le hpos.le
146   have : 0 < J w x * sinh x * sinh w := by rw [J_key]; linarith
147   have hm : 0 < sinh x * sinh w := mul_pos hsx hS
148   nlinarith
149
150 lemma J_neg {w x : ℝ} (hw : 0 < w) (hx : x < w) : J w x < 0 := by
151   have hS : 0 < sinh w := sinh_pos_iff.mpr hw
152   have hs : 0 < sinh (w / 2) := sinh_pos_iff.mpr (by linarith)
153   rcases lt_trichotomy x 0 with hx0 | hx0 | hx0
154   · rcases le_or_gt (-w) x with hxw | hxw
155     · -- −w ≤ x < 0
156       have h1 : sinh (x / 2) < 0 := sinh_neg_iff.mpr (by linarith)
157       have h2 : 0 ≤ sinh ((x + w) / 2) := sinh_nonneg_iff.mpr (by linarith)
158       have h3 : 0 < sinh (w / 2) * f x := mul_pos hs (f_pos x)
159       unfold J
160       nlinarith
161     · -- x < −w : write x = −z with z > w
162       obtain ⟨z, rfl⟩ : ∃ z, x = -z := ⟨-x, by ring⟩
163       have hzw : w < z := by linarith
164       have hz0 : 0 < z := hw.trans hzw
165       have hsz : 0 < sinh z := sinh_pos_iff.mpr hz0
166       have hL := L1 hw hzw
167       have e : f z * sinh w * sinh z = z * sinh w := by
168         linear_combination sinh w * f_mul_sinh z
169       have hfz : f z * sinh w < w := by nlinarith
170       have hsS : sinh (w / 2) < sinh w := sinh_lt_sinh.mpr (by linarith)
171       have hsf : sinh (w / 2) * f z < w := by
172         have := mul_lt_mul_of_pos_right hsS (f_pos z)
173         nlinarith
174       have hA : sinh ((-z + w) / 2) = -sinh ((z - w) / 2) := by
175         rw [← sinh_neg]; congr 1; ring
176       have hB : sinh (-z / 2) = -sinh (z / 2) := by
```

```
        rw [← sinh_neg]; congr 1; ring
      have hC0 : 0 < sinh ((z - w) / 2) := sinh_pos_iff.mpr (by linarith)
      have hC1 : sinh ((z - w) / 2) < sinh (z / 2) := sinh_lt_sinh.mpr (by linarith)
      have i1 := mul_lt_mul_of_pos_right hsf hC0
      have i2 := mul_lt_mul_of_pos_left hC1 hw
      unfold J
      rw [hA, hB, f_neg]
      nlinarith
  · subst hx0
    unfold J
    rw [f_zero]
    simp only [zero_div, sinh_zero, mul_zero, zero_add, mul_one]
    nlinarith
  · have hsx : 0 < sinh x := sinh_pos_iff.mpr hx0
    have hL := L1 hx0 hx
    have h2 := L2 (x / 2) (w / 2) (w / 2)
    rw [show w / 2 + w / 2 = w by ring, show x / 2 + w / 2 = (x + w) / 2 by ring] at h2
    have h2' : sinh (x / 2) * sinh w < sinh (w / 2) * sinh ((x + w) / 2) := by
      have : 0 < sinh (w / 2 - x / 2) * sinh (w / 2) :=
        mul_pos (sinh_pos_iff.mpr (by linarith)) hs
      linarith
    have hpos : 0 < sinh (x / 2) * sinh w := mul_pos (sinh_pos_iff.mpr (by linarith)) hS
    have hprod := mul_lt_mul'' hL h2' (mul_pos hw hsx).le hpos.le
    have : J w x * sinh x * sinh w < 0 := by rw [J_key]; linarith
    have hm : 0 < sinh x * sinh w := mul_pos hsx hS
    nlinarith

/-! ## Pure algebraic identities (certified by `linear_combination`) -/

/-- Eq. (S:detid): decomposition of the determinant of `M/(2q)`. Here
`P = κ cosh w − α`, `Q = κ cosh w − β` (so `P + Q = 2w` is the calibration). -/
lemma det_identity (κ κ0 w P Q S C sy cy sx cx F : ℝ)
    (hQ : Q = 2 * w - P) (hC : C ^ 2 - S ^ 2 = 1) (hy : cy ^ 2 - sy ^ 2 = 1)
    (hsx : sx = sy * C + cy * S) (hcx : cx = cy * C + sy * S) (h3 : κ0 * (C - 1) = w) :
    (κ * S * cy + P * sy + C * F) * (κ * S * cy + Q * sy + C * F) - (κ * S + F * cx) ^ 2
      = ((κ0 * S * cy + w * sy + C * F) ^ 2 - (κ0 * S + F * cx) ^ 2)
        + 2 * (κ - κ0) * sy * (w * sx - S ^ 2 * F)
        + ((κ * C - P) * (κ * C - Q) - κ ^ 2) * sy ^ 2 := by
  subst hQ hsx hcx
  linear_combination (sy ^ 2 * (κ0 - κ) * (κ + κ0)) * hC
    + (-S ^ 2 * (κ0 - κ) * (κ + κ0)) * hy + (-sy ^ 2 * (κ0 * (C + 1) - w)) * h3

/-- Half-angle form of the eigenvalue of `M/(2q)` along `(1,1)` at `κ = κ₀`, `δ = 0`. -/
lemma half_minus (κ0 w S C sy cy sxh cxh cx sxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (hsxw : sxw = sp * C + cp * S) :
    sr * ((κ0 * S * cy + w * sy + C * F) - (κ0 * S + F * cx))
      = 2 * sp * (w * sxh - sr * F * sxw) := by
  subst hw; subst hcx; subst hsxw; subst hC hS hsy hcy hsxh hcxh
  linear_combination (-sr * (F * cr ^ 2 + F * sr ^ 2 - 2 * cr * sr * κ0)) * hp
    + (-2 * cr * sp ^ 2 * κ0) * hr

/-- Half-angle form of the eigenvalue of `M/(2q)` along `(1,−1)` at `κ = κ₀`, `δ = 0`. -/
lemma half_plus (κ0 w S C sy cy sxh cxh cx sxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (_hsxw : sxw = sp * C + cp * S) :
    sr * ((κ0 * S * cy + w * sy + C * F) + (κ0 * S + F * cx))
      = 2 * w * cp * cxh + sr * F * (C + cx) := by
  subst hw; subst hcx; subst hC hS hsy hcy hsxh hcxh
  linear_combination (-2 * cr * κ0 * (cr - 1) * (cr + 1)) * hp
    + (-2 * cr * κ0 * (sp ^ 2 + 1)) * hr

/-! ## The matrix `M(x)` and its normalized form `M(x)/(2q)` -/

/-- Diagonal entry of `M(x)` for a general `q` (`a = α` or `a = β`). -/
def Mdiag (κ a q x : ℝ) : ℝ := q * (κ - q * a) * exp x + (a - q * κ) * exp (-x) + (1 + q ^ 2) * f x

/-- Off-diagonal entry of `M(x)` for a general `q`. -/
def Moff (κ q x : ℝ) : ℝ := -κ * (1 - q ^ 2) - 2 * q * b x

/-- Diagonal entry of `M(x)/(2q)`, `q = e^{−w}`: `m = κ S cosh(x−w) + (κC − a) sinh(x−w) + C f`. -/
def mdiag (κ a w x : ℝ) : ℝ :=
  κ * sinh w * cosh (x - w) + (κ * cosh w - a) * sinh (x - w) + cosh w * f x

/-- Minus the off-diagonal entry of `M(x)/(2q)`. -/
def moff (κ w x : ℝ) : ℝ := κ * sinh w + b x

lemma Mdiag_eq (κ a w x : ℝ) : Mdiag κ a (exp (-w)) x = 2 * exp (-w) * mdiag κ a w x := by
  unfold Mdiag mdiag
  simp only [cosh_eq, sinh_eq, Real.exp_neg, Real.exp_sub, neg_sub]
  field_simp
  ring

lemma Moff_eq (κ w x : ℝ) : Moff κ (exp (-w)) x = -(2 * exp (-w) * moff κ w x) := by
  unfold Moff moff
```

```
  simp only [sinh_eq, Real.exp_neg]
  field_simp
  ring

/-! ## Hypotheses of Lemma S5 -/

/-- Hypotheses of Lemma S5: `α, β, κ > 0`, `αβ ≥ κ²`, and `w > 0` solves the calibration
`κ cosh w − w = (α+β)/2` (then `q = e^{−w}`). -/
structure Hyp (κ α β w : ℝ) : Prop where
  hκ : 0 < κ
  hα : 0 < α
  hβ : 0 < β
  hw : 0 < w
  hκαβ : κ ^ 2 ≤ α * β
  hcal : κ * cosh w - w = (α + β) / 2

/-- Sanity check: the hypotheses are satisfiable (`w = 1`, `κ = 2/(cosh 1 − 1)`,
`α = β = κ cosh 1 − 1`). -/
theorem hyp_nonvacuous : ∃ κ α β w : ℝ, Hyp κ α β w := by
  have hC1 : 0 < cosh 1 - 1 := by have := one_lt_cosh.mpr (one_ne_zero (α := ℝ)); linarith
  set κ := 2 / (cosh 1 - 1) with hκ
  have hκpos : 0 < κ := div_pos two_pos hC1
  have hk : κ * (cosh 1 - 1) = 2 := div_mul_cancel₀ 2 hC1.ne'
  refine (κ, κ * cosh 1 - 1, 1, hκpos, by nlinarith, by nlinarith, one_pos,
    by nlinarith, by ring)

variable {κ α β w : ℝ}

lemma kappa0_le (h : Hyp κ α β w) : w / (cosh w - 1) ≤ κ := by
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr h.hw.ne'; linarith
  have h1 : (2 * κ) ^ 2 ≤ (α + β) ^ 2 := by nlinarith [sq_nonneg (α - β), h.hκαβ]
  have hab : 2 * κ ≤ α + β := by nlinarith [h.hκ, h.hα, h.hβ]
  rw [div_le_iff₀ hC1]
  linarith [h.hcal]

/-- `κ sinh w > w`. -/
lemma kS_gt (h : Hyp κ α β w) : w < κ * sinh w := by
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr h.hw.ne'; linarith
  have hS : 0 < sinh w := sinh_pos_iff.mpr h.hw
  have hSC : cosh w - 1 < sinh w := by
    have h1 := cosh_sub_sinh w
    have h2 : exp (-w) < 1 := Real.exp_lt_one_iff.mpr (by linarith [h.hw])
    have h3 : 0 < exp (-w) := exp_pos _
    linarith
  have hk := kappa0_le h
  rw [div_le_iff₀ hC1] at hk
  nlinarith [h.hκ]

/-- The trace of `M(x)/(2q)` is positive. -/
lemma trace_pos (h : Hyp κ α β w) (x : ℝ) : 0 < mdiag κ α w x + mdiag κ β w x := by
  have hk := kS_gt h
  have hcy : 0 < cosh (x - w) := cosh_pos _
  have he : 0 < cosh (x - w) + sinh (x - w) := by rw [cosh_add_sinh]; exact exp_pos _
  have hCf : 0 < cosh w * f x := mul_pos (cosh_pos _) (f_pos x)
  have hsum : mdiag κ α w x + mdiag κ β w x
      = 2 * (κ * sinh w - w) * cosh (x - w) + 2 * w * (cosh (x - w) + sinh (x - w))
        + 2 * (cosh w * f x) := by
    unfold mdiag; linear_combination (2 * sinh (x - w)) * h.hcal
  rw [hsum]
  have : 0 < (κ * sinh w - w) * cosh (x - w) := mul_pos (by linarith) hcy
  nlinarith [h.hw]

/-- At `x = w` all four entries of `M(w)/(2q)` agree (up to the sign of the off-diagonal). -/
lemma at_w_alpha (κ a w : ℝ) : mdiag κ a w w = moff κ w w := by
  unfold mdiag moff b
  simp only [sub_self, cosh_zero, sinh_zero]
  ring

set_option maxHeartbeats 1000000 in
/-- Main step: `det(M(x)/(2q)) > 0` for `x ≠ w`. -/
theorem det_pos (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) :
    0 < mdiag κ α w x * mdiag κ β w x - moff κ w x ^ 2 := by
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr h.hw.ne'; linarith
  set κ0 := w / (cosh w - 1) with hκ0def
  have h3 : κ0 * (cosh w - 1) = w := div_mul_cancel₀ w hC1.ne'
  have hk : κ0 ≤ κ := kappa0_le h
  have hdet := det_identity κ κ0 w (κ * cosh w - α) (κ * cosh w - β) (sinh w) (cosh w)
    (sinh (x - w)) (cosh (x - w)) (sinh x) (cosh x) (f x)
    (by linarith [h.hcal]) (by rw [cosh_sq]; ring) (by rw [cosh_sq]; ring)
    (by rw [← sinh_add, sub_add_cancel]) (by rw [← cosh_add, sub_add_cancel]) h3
  -- half-angle data
  have hp : cosh ((x - w) / 2) ^ 2 - sinh ((x - w) / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hr : cosh (w / 2) ^ 2 - sinh (w / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hCr : cosh w = cosh (w / 2) ^ 2 + sinh (w / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hSr : sinh w = 2 * sinh (w / 2) * cosh (w / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hsy : sinh (x - w) = 2 * sinh ((x - w) / 2) * cosh ((x - w) / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hcy : cosh (x - w) = cosh ((x - w) / 2) ^ 2 + sinh ((x - w) / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
```

```
  have hsxh : sinh (x / 2) = sinh ((x - w) / 2) * cosh (w / 2) + cosh ((x - w) / 2) * sinh (w / 2) := by
    rw [← sinh_add]; congr 1; ring
  have hcxh : cosh (x / 2) = cosh ((x - w) / 2) * cosh (w / 2) + sinh ((x - w) / 2) * sinh (w / 2) := by
    rw [← cosh_add]; congr 1; ring
  have hcx : cosh x = cosh (x / 2) ^ 2 + sinh (x / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hsxw : sinh ((x + w) / 2) = sinh ((x - w) / 2) * cosh w + cosh ((x - w) / 2) * sinh w := by
    rw [← sinh_add]; congr 1; ring
  have hm := half_minus κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (sinh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hsxw
  have hpl := half_plus κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (sinh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hsxw
  have hsr : 0 < sinh (w / 2) := sinh_pos_iff.mpr (by linarith [h.hw])
  -- sign facts on either side of x = w
  have hsJ : 0 < sinh ((x - w) / 2) * J w x := by
    rcases lt_or_gt_of_ne hx with hlt | hgt
    · exact mul_pos_of_neg_of_neg (sinh_neg_iff.mpr (by linarith)) (J_neg h.hw hlt)
    · exact mul_pos (sinh_pos_iff.mpr (by linarith)) (J_pos h.hw hgt)
  have hsg : 0 ≤ sinh (x - w) * (w * sinh x - sinh w ^ 2 * f x) := by
    rcases lt_or_gt_of_ne hx with hlt | hgt
    · exact (mul_pos_of_neg_of_neg (sinh_neg_iff.mpr (by linarith)) (g_neg h.hw hlt)).le
    · exact (mul_pos (sinh_pos_iff.mpr (by linarith)) (g_pos h.hw hgt)).le
  -- the two eigenvalues at κ = κ₀, δ = 0
  have hAm : 0 < (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      - (κ0 * sinh w + f x * cosh x) := by
    have e1 : 0 < sinh (w / 2) * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      - (κ0 * sinh w + f x * cosh x)) := by
      rw [hm]; unfold J at hsJ; linarith
    exact (mul_pos_iff_of_pos_left hsr).mp e1
  have hAp : 0 < (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      + (κ0 * sinh w + f x * cosh x) := by
    have q1 : 0 < 2 * w * cosh ((x - w) / 2) * cosh (x / 2) :=
      mul_pos (mul_pos (by linarith [h.hw]) (cosh_pos _)) (cosh_pos _)
    have q2 : 0 < sinh (w / 2) * f x * (cosh w + cosh x) :=
      mul_pos (mul_pos hsr (f_pos x)) (by linarith [cosh_pos w, cosh_pos x])
    have e1 : 0 < sinh (w / 2) * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
      + (κ0 * sinh w + f x * cosh x)) := by rw [hpl]; linarith
    exact (mul_pos_iff_of_pos_left hsr).mp e1
  have hΔ0 : 0 < (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x) ^ 2
      - (κ0 * sinh w + f x * cosh x) ^ 2 := by
    have e : (κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x) ^ 2
        - (κ0 * sinh w + f x * cosh x) ^ 2
        = ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
          - (κ0 * sinh w + f x * cosh x))
          * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
          + (κ0 * sinh w + f x * cosh x)) := by ring
    rw [e]; exact mul_pos hAm hAp
  have hT2 : 0 ≤ 2 * (κ - κ0) * sinh (x - w) * (w * sinh x - sinh w ^ 2 * f x) := by
    have : 0 ≤ 2 * (κ - κ0) := by linarith
    have := mul_nonneg this hsg
    linarith
  have hT3 : 0 ≤ ((κ * cosh w - (κ * cosh w - α)) * (κ * cosh w - (κ * cosh w - β)) - κ ^ 2)
      * sinh (x - w) ^ 2 := by
    have : (κ * cosh w - (κ * cosh w - α)) * (κ * cosh w - (κ * cosh w - β)) - κ ^ 2
        = α * β - κ ^ 2 := by ring
    rw [this]
    exact mul_nonneg (by linarith [h.hκαβ]) (sq_nonneg _)
  unfold mdiag moff b
  linarith [hdet, hΔ0, hT2, hT3]

/-- `det(M(x)/(2q)) ≥ 0` for every real `x`. -/
theorem det_nonneg (h : Hyp κ α β w) (x : ℝ) :
    0 ≤ mdiag κ α w x * mdiag κ β w x - moff κ w x ^ 2 := by
  by_cases hx : x = w
  · rw [hx, at_w_alpha, at_w_alpha]; nlinarith [sq_nonneg (moff κ w w)]
  · exact (det_pos h hx).le

lemma diag_nonneg (h : Hyp κ α β w) (x : ℝ) :
    0 ≤ mdiag κ α w x ∧ 0 ≤ mdiag κ β w x := by
  have hd := det_nonneg h x
  have ht := trace_pos h x
  constructor <;> nlinarith [sq_nonneg (moff κ w x), sq_nonneg (mdiag κ α w x), sq_nonneg (mdiag κ β w x)]

/-! ## Quadratic forms of 2×2 symmetric matrices -/

lemma quad_nonneg {a c d : ℝ} (ha : 0 ≤ a) (hdet : d ^ 2 ≤ a * c) (hc : 0 ≤ c) (u v : ℝ) :
    0 ≤ a * u ^ 2 - 2 * d * u * v + c * v ^ 2 := by
  rcases ha.eq_or_lt with h0 | hpos
  · subst h0
    have hd : d = 0 := by
      have : d ^ 2 ≤ 0 := by linarith
      exact pow_eq_zero_iff (n := 2) (by norm_num) |>.mp (le_antisymm this (sq_nonneg d))
    subst hd
    nlinarith [sq_nonneg v]
  · nlinarith [sq_nonneg (a * u - d * v), mul_nonneg (sub_nonneg.2 hdet) (sq_nonneg v)]

lemma quad_pos {a c d : ℝ} (ha : 0 < a) (hdet : d ^ 2 < a * c) {u v : ℝ}
    (huv : u ≠ 0 ∨ v ≠ 0) : 0 < a * u ^ 2 - 2 * d * u * v + c * v ^ 2 := by
  by_cases hv : v = 0
```

```
  · subst hv
    have hu : u ≠ 0 := by tauto
    have : 0 < u ^ 2 := by positivity
    nlinarith
  · have : 0 < v ^ 2 := by positivity
    nlinarith [sq_nonneg (a * u - d * v), mul_pos (sub_pos.2 hdet) this]

/-! ## Lemma S5 (positivity and kernel of `M(x)`) -/

/-- The quadratic form of `M(x)`, `q = e^{-w}`. -/
def QM (κ α β w x u v : ℝ) : ℝ :=
  Mdiag κ α (exp (-w)) x * u ^ 2 + 2 * Moff κ (exp (-w)) x * u * v + Mdiag κ β (exp (-w)) x * v ^ 2

lemma QM_eq (κ α β w x u v : ℝ) : QM κ α β w x u v
    = 2 * exp (-w) * (mdiag κ α w x * u ^ 2 - 2 * moff κ w x * u * v + mdiag κ β w x * v ^ 2) := by
  unfold QM; rw [Mdiag_eq, Mdiag_eq, Moff_eq]; ring

/-- **Lemma S5 (i)**: `M(x) ≥ 0` for every real `x`. -/
theorem lemmaS5_psd (h : Hyp κ α β w) (x u v : ℝ) : 0 ≤ QM κ α β w x u v := by
  rw [QM_eq]
  have ⟨h1, h2⟩ := diag_nonneg h x
  have hd := det_nonneg h x
  exact mul_nonneg (by positivity) (quad_nonneg h1 (by linarith) h2 u v)

/-- **Lemma S5 (ii)**: `M(x)` is positive definite for `x ≠ w`, i.e. `ker M(x) = {0}`. -/
theorem lemmaS5_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {u v : ℝ} (huv : u ≠ 0 ∨ v ≠ 0) :
    0 < QM κ α β w x u v := by
  rw [QM_eq]
  have hd := det_pos h hx
  have ⟨h1, h2⟩ := diag_nonneg h x
  have ha : 0 < mdiag κ α w x := by
    rcases h1.eq_or_lt with h0 | h0
    · rw [← h0] at hd; nlinarith [sq_nonneg (moff κ w x)]
    · exact h0
  exact mul_pos (by positivity) (quad_pos ha (by linarith) huv)

/-- **Lemma S5 (iii)**: at `x = w`, the quadratic form vanishes exactly on `span{(1,1)}`,
i.e. `ker M(w) = span{(1,1)}`. -/
theorem lemmaS5_kernel (h : Hyp κ α β w) (u v : ℝ) : QM κ α β w w u v = 0 ↔ u = v := by
  rw [QM_eq, at_w_alpha, at_w_alpha]
  have ht := trace_pos h w
  rw [at_w_alpha, at_w_alpha] at ht
  have hq : 0 < 2 * exp (-w) := by positivity
  have hc : 0 < moff κ w w := by linarith
  have e : moff κ w w * u ^ 2 - 2 * moff κ w w * u * v + moff κ w w * v ^ 2
      = moff κ w w * (u - v) ^ 2 := by ring
  rw [e]
  constructor
  · intro h0
    have : moff κ w w * (u - v) ^ 2 = 0 := by
      rcases mul_eq_zero.mp h0 with h1 | h1
      · linarith
      · exact h1
    rcases mul_eq_zero.mp this with h1 | h1
    · linarith
    · have := pow_eq_zero_iff (n := 2) (by norm_num) |>.mp h1
      linarith
  · intro huv; subst huv; ring

/-- `M(w) (1,1)ᵀ = 0`, entrywise. -/
theorem lemmaS5_null :
    Mdiag κ α (exp (-w)) w + Moff κ (exp (-w)) w = 0 ∧
    Moff κ (exp (-w)) w + Mdiag κ β (exp (-w)) w = 0 := by
  rw [Mdiag_eq, Mdiag_eq, Moff_eq, at_w_alpha, at_w_alpha]
  constructor <;> ring

/-- Complex form used in Eq. (S:mainid): for `z = (A, conj B)`,
`z† M(x) z = M₁₁|A|² + M₂₂|B|² + 2 M₁₂ Re(AB) ≥ 0`. -/
theorem lemmaS5_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    0 ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re := by
  have hpsd := lemmaS5_psd h x ‖A‖ ‖B‖
  unfold QM at hpsd
  have hoff : Moff κ (exp (-w)) x ≤ 0 := by
    rw [Moff_eq]
    have : 0 < moff κ w x := by
      unfold moff b
      have := kS_gt h
      have : 0 < f x * cosh x := mul_pos (f_pos x) (cosh_pos x)
      nlinarith [h.hw]
    have : 0 < exp (-w) := exp_pos _
    nlinarith
  have hre : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  nlinarith [mul_le_mul_of_nonpos_left hre hoff]

/-- The off-diagonal entry is negative: `M₁₂(x) < 0`. -/
lemma Moff_neg (h : Hyp κ α β w) (x : ℝ) : Moff κ (exp (-w)) x < 0 := by
  rw [Moff_eq]
  have : 0 < moff κ w x := by
```

```
    unfold moff b
    have := kS_gt h
    have : 0 < f x * cosh x := mul_pos (f_pos x) (cosh_pos x)
    nlinarith [h.hw]
  have : 0 < exp (-w) := exp_pos _
  nlinarith

/-- Complex strict form: for `x ≠ w` and `z = (A, conj B) ≠ 0`, `z† M(x) z > 0`. -/
theorem lemmaS5_complex_pd (h : Hyp κ α β w) {x : ℝ} (hx : x ≠ w) {A B : ℂ}
    (hAB : A ≠ 0 ∨ B ≠ 0) :
    0 < Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
      + 2 * Moff κ (exp (-w)) x * (A * B).re := by
  have huv : ‖A‖ ≠ 0 ∨ ‖B‖ ≠ 0 := by
    rcases hAB with hA | hB
    · exact Or.inl (norm_ne_zero_iff.mpr hA)
    · exact Or.inr (norm_ne_zero_iff.mpr hB)
  have hpd := lemmaS5_pd h hx huv
  unfold QM at hpd
  have hoff := (Moff_neg h x).le
  have hre : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  nlinarith [mul_le_mul_of_nonpos_left hre hoff]

/-- At `x = w`: `z† M(w) z = 2q (κ sinh w + b(w)) |A − conj B|²`. -/
theorem lemmaS5_complex_at_w (A B : ℂ) :
    Mdiag κ α (exp (-w)) w * Complex.normSq A + Mdiag κ β (exp (-w)) w * Complex.normSq B
      + 2 * Moff κ (exp (-w)) w * (A * B).re
      = 2 * exp (-w) * moff κ w w * Complex.normSq (A - (starRingEnd ℂ) B) := by
  rw [Mdiag_eq, Mdiag_eq, Moff_eq, at_w_alpha, at_w_alpha]
  simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.conj_re,
    Complex.conj_im, Complex.mul_re]
  ring

/-- Complex kernel at `x = w`: `z† M(w) z = 0` iff `z = (A, conj B)` lies in `span{(1,1)}`. -/
theorem lemmaS5_complex_kernel (h : Hyp κ α β w) (A B : ℂ) :
    Mdiag κ α (exp (-w)) w * Complex.normSq A + Mdiag κ β (exp (-w)) w * Complex.normSq B
      + 2 * Moff κ (exp (-w)) w * (A * B).re = 0 ↔ A = (starRingEnd ℂ) B := by
  rw [lemmaS5_complex_at_w]
  have ht := trace_pos h w
  rw [at_w_alpha, at_w_alpha] at ht
  have hc : 0 < 2 * exp (-w) * moff κ w w := mul_pos (by positivity) (by linarith)
  rw [mul_eq_zero, Complex.normSq_eq_zero, sub_eq_zero]
  constructor
  · rintro (h0 | h0)
    · linarith
    · exact h0
  · intro h0; exact Or.inr h0

lemma Hyp.swap (h : Hyp κ α β w) : Hyp κ β α w :=
  ⟨h.hκ, h.hβ, h.hα, h.hw, by linarith [h.hκαβ, mul_comm α β], by linarith [h.hcal]⟩

/-- `q κ < α < κ / q` (written as `q α < κ`), with `q = e^{−w}`. -/
lemma alpha_bounds (h : Hyp κ α β w) : exp (-w) * κ < α ∧ exp (-w) * α < κ := by
  have hE : exp w * exp (-w) = 1 := by rw [← exp_add]; simp
  have hEe : exp (-w) < exp w := exp_lt_exp.mpr (by linarith [h.hw])
  have he : 0 < exp (-w) := exp_pos _
  have hβ : β = κ * (exp w + exp (-w)) - 2 * w - α := by
    have := h.hcal; rw [cosh_eq] at this; linarith
  have hk := h.hκαβ
  rw [hβ] at hk
  have hwα : 0 < w * α := mul_pos h.hw h.hα
  have hE2 : κ ^ 2 * (exp w * exp (-w)) = κ ^ 2 := by rw [hE, mul_one]
  constructor
  · by_contra hc
    replace hc := not_lt.mp hc
    have h1 : 0 ≤ (exp (-w) * κ - α) * (exp w * κ - α) :=
      mul_nonneg (by linarith) (by nlinarith [h.hκ])
    nlinarith
  · by_contra hc
    replace hc := not_lt.mp hc
    have hα' : exp w * κ ≤ α := by
      have h1 := mul_le_mul_of_nonneg_left hc (exp_pos w).le
      have h2 : exp w * (exp (-w) * α) = α := by rw [← mul_assoc, hE, one_mul]
      linarith
    have h1 : 0 ≤ (α - exp w * κ) * (α - exp (-w) * κ) :=
      mul_nonneg (by linarith) (by nlinarith [h.hκ])
    nlinarith

end Thm2Cert
```

Thm2Cert/Identity.lean

```
/-
Certificate for the exact identity Eq. (S:mainid) of the Supplemental Material:
adding the two second variations Q(O₁ψ), Q(O₂ψ) of Eq. (S:Q12), and using the sum rules
Eq. (S:wccr), gives  −∑_{l,r} c_l c_r z†_{lr} M(x_{lr}) z_{lr}.

Input (taken as hypotheses / definitions, NOT derived here): the expressions (S:Q12) for the
```

```
second variations, and the sum rules ⟨N⟩ = ∑ p_r|A_lr|², ⟨N⟩+1 = ∑ p_l|A_lr|².
Setting: a finite index set. `mainid` is Eq. (S:mainid) verbatim, for Schmidt coefficients
c_l > 0, with p_l = c_l², L_lr = (ln p_l − ln p_r)/(p_l − p_r) (= 1/p_l if p_l = p_r),
x_lr = ln(c_l/c_r). `mainid_general` / `sumQ_nonpos_general` allow c_l ≥ 0, so that kernel
directions (support-to-kernel entries, retained in the text as continuous limits) are included;
this covers every state of finite Schmidt rank. Infinite Schmidt rank (the limiting argument of
the SM) is not covered.
-/
import Thm2Cert.MatrixLemma
import Mathlib.Analysis.SpecialFunctions.Log.Basic

open Real Finset

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-- The logarithmic mean kernel `L(p,p') = (ln p − ln p')/(p − p')`, `= 1/p` on the diagonal. -/
def Lker (p p' : ℝ) : ℝ := if p = p' then 1 / p else (log p - log p') / (p - p')

section scalar

variable {cl cr : ℝ}

lemma E1 (hl : 0 < cl) (hr : 0 < cr) : cl * cr * exp (log (cl / cr)) = cl ^ 2 := by
  rw [exp_log (div_pos hl hr)]; field_simp

lemma E2 (hl : 0 < cl) (hr : 0 < cr) : cl * cr * exp (-log (cl / cr)) = cr ^ 2 := by
  rw [exp_neg, exp_log (div_pos hl hr)]; field_simp

lemma E3 (hl : 0 < cl) (hr : 0 < cr) :
    cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2) = cl * cr * f (log (cl / cr)) := by
  unfold Lker
  by_cases h : cl ^ 2 = cr ^ 2
  · have hlr : cl = cr := (pow_left_inj₀ hl.le hr.le two_ne_zero).mp h
    subst hlr
    simp only [↓reduceIte]
    rw [div_self hl.ne', log_one, f_zero]
    field_simp
  · simp only [h, ↓reduceIte]
    have hlr : cl ≠ cr := fun e => h (by rw [e])
    have hq : cl / cr ≠ 1 := by
      intro e; apply hlr; field_simp at e; linarith
    have hX : log (cl / cr) ≠ 0 := log_ne_zero_of_pos_of_ne_one (div_pos hl hr) hq
    unfold f
    simp only [hX, ↓reduceIte]
    rw [sinh_eq, exp_neg, exp_log (div_pos hl hr), log_pow, log_pow,
      log_div hl.ne' hr.ne']
    have h2 : cl ^ 2 - cr ^ 2 ≠ 0 := sub_ne_zero.mpr h
    have h3 : cl / cr - (cl / cr)⁻¹ ≠ 0 := by
      intro e
      apply h2
      field_simp at e
      linarith
    field_simp
    ring

lemma E3' (hl : 0 < cl) (hr : 0 < cr) :
    cl * cr * Lker (cl ^ 2) (cr ^ 2) = f (log (cl / cr)) := by
  have h := E3 hl hr
  have hc : 0 < cl * cr := mul_pos hl hr
  have : cl * cr * (cl * cr * Lker (cl ^ 2) (cr ^ 2)) = cl * cr * f (log (cl / cr)) := by
    rw [← h]; ring
  exact mul_left_cancel₀ hc.ne' this

lemma E4 (hl : 0 < cl) (hr : 0 < cr) :
    √(cl ^ 2 * cr ^ 2) * (cl ^ 2 + cr ^ 2) * Lker (cl ^ 2) (cr ^ 2)
      = 2 * cl * cr * b (log (cl / cr)) := by
  have hs : √(cl ^ 2 * cr ^ 2) = cl * cr := by
    rw [show cl ^ 2 * cr ^ 2 = (cl * cr) * (cl * cr) by ring]
    exact sqrt_mul_self (mul_pos hl hr).le
  rw [hs]
  unfold b
  rw [← E3' hl hr, cosh_eq, exp_neg, exp_log (div_pos hl hr)]
  field_simp

end scalar

variable {ι : Type*} [Fintype ι]

/-- `k = Re(a b) = ∑ c_l c_r Re(A_lr B_lr)`. -/
def kk (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ := ∑ l, ∑ r, c l * c r * (A l r * B l r).re

/-- `A = ∑ p_l p_r L_lr |A_lr|²` (and `B` with `B`). -/
def sfA (c : ι → ℝ) (A : ι → ι → ℂ) : ℝ :=
  ∑ l, ∑ r, c l ^ 2 * c r ^ 2 * Lker (c l ^ 2) (c r ^ 2) * Complex.normSq (A l r)

/-- `C = ∑ √(p_l p_r) (p_l + p_r) L_lr Re(A_lr B_lr)`. -/
def sfC (c : ι → ℝ) (A B : ι → ι → ℂ) : ℝ :=
```

```
  ∑ l, ∑ r, √(c l ^ 2 * c r ^ 2) * (c l ^ 2 + c r ^ 2) * Lker (c l ^ 2) (c r ^ 2) * (A l r * B l r).re

/-- Eq. (S:Q12), first line: `Q(O₁ψ)`. -/
def Q1 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q * κ - α) * nA + (q ^ 2 * β - q * κ) * (nB + 1) + (κ * (1 - q ^ 2) + q * (α - β)) * k
    - sA - q ^ 2 * sB + q * sC

/-- Eq. (S:Q12), second line: `Q(O₂ψ)`. -/
def Q2 (α β κ q nA nB k sA sB sC : ℝ) : ℝ :=
  (q ^ 2 * α - q * κ) * (nA + 1) + (q * κ - β) * nB + (κ * (1 - q ^ 2) - q * (α - β)) * k
    - q ^ 2 * sA - sB + q * sC

/-- **Eq. (S:mainid)** (exact bookkeeping identity), for an arbitrary real `q`. -/
theorem mainid (c : ι → ℝ) (hc : ∀ l, 0 < c l) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
    = -∑ l, ∑ r, c l * c r *
        (Mdiag κ α q (log (c l / c r)) * Complex.normSq (A l r)
          + Mdiag κ β q (log (c l / c r)) * Complex.normSq (B l r)
          + 2 * Moff κ q (log (c l / c r)) * (A l r * B l r).re) := by
  have step : Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      = (q * κ - α) * nA + (q ^ 2 * α - q * κ) * (nA + 1)
        + (q ^ 2 * β - q * κ) * (nB + 1) + (q * κ - β) * nB
        + 2 * κ * (1 - q ^ 2) * kk c A B - (1 + q ^ 2) * sfA c A - (1 + q ^ 2) * sfA c B
        + 2 * q * sfC c A B := by
    unfold Q1 Q2; ring
  rw [step, hA1, hB1, hA0, hB0]
  unfold kk sfA sfC
  simp only [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib,
    ← Finset.sum_neg_distrib]
  refine Finset.sum_congr rfl (fun l _ => Finset.sum_congr rfl (fun r _ => ?_))
  have e1 := E1 (hc l) (hc r)
  have e2 := E2 (hc l) (hc r)
  have e3 := E3 (hc l) (hc r)
  have e4 := E4 (hc l) (hc r)
  unfold Mdiag Moff
  linear_combination
    ((α - q * κ) * Complex.normSq (A l r) + (β - q * κ) * Complex.normSq (B l r)) * e2
    + (q * (κ - q * α) * Complex.normSq (A l r) + q * (κ - q * β) * Complex.normSq (B l r)) * e1
    - (1 + q ^ 2) * (Complex.normSq (A l r) + Complex.normSq (B l r)) * e3
    + 2 * q * (A l r * B l r).re * e4

/-- **Consequence**: with `q = e^{-w}` and the hypotheses of Lemma S5,
`Q(O₁ψ) + Q(O₂ψ) ≤ 0`. -/
theorem sumQ_nonpos {κ α β w : ℝ} (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 < c l)
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) ≤ 0 := by
  rw [mainid c hc A B α β κ (exp (-w)) nA nB hA0 hA1 hB0 hB1, neg_nonpos]
  refine Finset.sum_nonneg (fun l _ => Finset.sum_nonneg (fun r _ => ?_))
  exact mul_nonneg (mul_pos (hc l) (hc r)).le (lemmaS5_complex h _ (A l r) (B l r))

/-! ## The identity for arbitrary `c_l ≥ 0` (kernel directions included)

For a state of finite Schmidt rank, the sum rules (S:wccr) also involve pairs `(l, r)` with
one vanishing Schmidt coefficient (support-to-kernel entries of `a`, `b`). Indexing the support
together with a finite set of kernel directions (`c_l = 0`) covers this case. Here the
coefficients are written through `p_l = c_l²`, `Φ_lr = p_l p_r L_lr` and
`Ψ_lr = √(p_l p_r)(p_l + p_r) L_lr`, which equal `0` when `p_l p_r = 0`
(the "continuous limits" of the text). -/

/-- Diagonal weight `c_l c_r M₁₁(x_lr)` written in the `p`-variables. -/
def wdiag (κ a q pl pr Φ : ℝ) : ℝ := q * (κ - q * a) * pl + (a - q * κ) * pr + (1 + q ^ 2) * Φ

/-- Off-diagonal weight `c_l c_r M₁₂(x_lr)` written in the `p`-variables. -/
def woff (κ q cc Ψ : ℝ) : ℝ := -κ * (1 - q ^ 2) * cc - q * Ψ

/-- The summand of Eq. (S:mainid), for Schmidt coefficients `cl, cr ≥ 0`. -/
def term (κ α β q cl cr : ℝ) (A B : ℂ) : ℝ :=
  wdiag κ α q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq A
  + wdiag κ β q (cl ^ 2) (cr ^ 2) (cl ^ 2 * cr ^ 2 * Lker (cl ^ 2) (cr ^ 2)) * Complex.normSq B
  + 2 * woff κ q (cl * cr) (√(cl ^ 2 * cr ^ 2) * (cl ^ 2 + cr ^ 2) * Lker (cl ^ 2) (cr ^ 2))
      * (A * B).re

/-- **Eq. (S:mainid), general form**: valid for all real `c_l` (no positivity needed). -/
theorem mainid_general (c : ι → ℝ) (A B : ι → ι → ℂ) (α β κ q nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
```

```
    = -∑ l, ∑ r, term κ α β q (c l) (c r) (A l r) (B l r) := by
  have step : Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      = (q * κ - α) * nA + (q ^ 2 * α - q * κ) * (nA + 1)
        + (q ^ 2 * β - q * κ) * (nB + 1) + (q * κ - β) * nB
        + 2 * κ * (1 - q ^ 2) * kk c A B - (1 + q ^ 2) * sfA c A - (1 + q ^ 2) * sfA c B
        + 2 * q * sfC c A B := by
    unfold Q1 Q2; ring
  rw [step, hA1, hB1, hA0, hB0]
  unfold kk sfA sfC
  simp only [Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib,
    ← Finset.sum_neg_distrib]
  refine Finset.sum_congr rfl (fun l _ => Finset.sum_congr rfl (fun r _ => ?_))
  unfold term wdiag woff
  ring

/-- Each summand is nonnegative under the hypotheses of Lemma S5, for all `cl, cr ≥ 0`:
for `cl, cr > 0` it is `c_l c_r z†  M(x_lr)  z` (Lemma S5); if one coefficient vanishes it
reduces to the boundary weights, nonnegative because `qκ < α, β < κ/q`. -/
theorem term_nonneg {κ α β w : ℝ} (h : Hyp κ α β w) {cl cr : ℝ} (hl : 0 ≤ cl) (hr : 0 ≤ cr)
    (A B : ℂ) : 0 ≤ term κ α β (exp (-w)) cl cr A B := by
  have ⟨ha1, ha2⟩ := alpha_bounds h
  have ⟨hb1, hb2⟩ := alpha_bounds h.swap
  have he : 0 < exp (-w) := exp_pos _
  have nA := Complex.normSq_nonneg A
  have nB := Complex.normSq_nonneg B
  rcases hl.eq_or_lt with hl0 | hlp
  · subst hl0
    unfold term wdiag woff
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, zero_add, add_zero, sub_zero]
    have : 0 ≤ (α - exp (-w) * κ) * cr ^ 2 * Complex.normSq A :=
      mul_nonneg (mul_nonneg (by linarith) (sq_nonneg _)) nA
    have : 0 ≤ (β - exp (-w) * κ) * cr ^ 2 * Complex.normSq B :=
      mul_nonneg (mul_nonneg (by linarith) (sq_nonneg _)) nB
    nlinarith
  rcases hr.eq_or_lt with hr0 | hrp
  · subst hr0
    unfold term wdiag woff
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, add_zero, sub_zero]
    have : 0 ≤ exp (-w) * (κ - exp (-w) * α) * cl ^ 2 * Complex.normSq A :=
      mul_nonneg (mul_nonneg (mul_nonneg he.le (by linarith)) (sq_nonneg _)) nA
    have : 0 ≤ exp (-w) * (κ - exp (-w) * β) * cl ^ 2 * Complex.normSq B :=
      mul_nonneg (mul_nonneg (mul_nonneg he.le (by linarith)) (sq_nonneg _)) nB
    nlinarith
  · have e1 := E1 hlp hrp
    have e2 := E2 hlp hrp
    have e3 := E3 hlp hrp
    have e4 := E4 hlp hrp
    have key : term κ α β (exp (-w)) cl cr A B = cl * cr *
        (Mdiag κ α (exp (-w)) (log (cl / cr)) * Complex.normSq A
          + Mdiag κ β (exp (-w)) (log (cl / cr)) * Complex.normSq B
          + 2 * Moff κ (exp (-w)) (log (cl / cr)) * (A * B).re) := by
      unfold term wdiag woff Mdiag Moff
      linear_combination
        (-exp (-w) * (κ - exp (-w) * α) * Complex.normSq A
          - exp (-w) * (κ - exp (-w) * β) * Complex.normSq B) * e1
        + (-(α - exp (-w) * κ) * Complex.normSq A - (β - exp (-w) * κ) * Complex.normSq B) * e2
        + (1 + exp (-w) ^ 2) * (Complex.normSq A + Complex.normSq B) * e3
        - 2 * exp (-w) * (A * B).re * e4
    rw [key]
    exact mul_nonneg (mul_pos hlp hrp).le (lemmaS5_complex h _ A B)

/-- **Consequence, general form**: under the hypotheses of Lemma S5, `Q(O₁ψ) + Q(O₂ψ) ≤ 0`
for all Schmidt coefficients `c_l ≥ 0` (finite index set). -/
theorem sumQ_nonpos_general {κ α β w : ℝ} (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l)
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) ≤ 0 := by
  rw [mainid_general c A B α β κ (exp (-w)) nA nB hA0 hA1 hB0 hB1, neg_nonpos]
  exact Finset.sum_nonneg (fun l _ => Finset.sum_nonneg (fun r _ =>
    term_nonneg h (hc l) (hc r) (A l r) (B l r)))

end Thm2Cert
```

Thm2Cert/Reduction.lean

```
/-
Certificate for the scalar steps connecting the minimum principle (Theorem S4 in the SM)
to Theorem 2 of the Letter:
  * the calibration κ cosh w − w = (α+β)/2 with (α,β,κ) = λ(t²,1,t), q = e^{−w},
    is equivalent to λ = λ_{q,t} = −2q ln q / [(t−q)(1−tq)], and λ_{q,t} > 0;
  * E(q) = g(q²/(1−q²)) = −ln(1−q²) − 2 n_q ln q;
  * the equality value on the two-mode squeezed vacuum:
```

```
      E(q) + (α+β) n_q − 2κ q/(1−q²) = −ln(1−q²) − κ q   (under the calibration);
  * E(q) + λ_{q,t} d_t(q) = −ln(1−q²) − λ_{q,t} t q + λ_{q,t};
  * the m-pair inequality of Theorem 2 follows from the minimum principle.
The TMSV moments ⟨N_A⟩ = ⟨N_B⟩ = q²/(1−q²), ⟨ab⟩ = q/(1−q²), and the operator identity
λ D̂_t = T + λ, are inputs (standard), not derived here.
-/
import Thm2Cert.MatrixLemma
import Mathlib.Analysis.SpecialFunctions.Log.Basic

open Real Finset

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-- `λ_{q,t} = −2 q ln q / [(t − q)(1 − t q)]`. -/
def lam (q t : ℝ) : ℝ := -2 * q * log q / ((t - q) * (1 - t * q))

/-- `d_t(q) = (1 − t q)² / (1 − q²)`. -/
def dt (t q : ℝ) : ℝ := (1 - t * q) ^ 2 / (1 - q ^ 2)

/-- `g(y) = (y+1) ln(y+1) − y ln y`. -/
def gfun (y : ℝ) : ℝ := (y + 1) * log (y + 1) - y * log y

/-- `E(q) = g(q²/(1−q²))`, the entanglement of the TMSV. -/
def EE (q : ℝ) : ℝ := gfun (q ^ 2 / (1 - q ^ 2))

lemma calib_factor (w t : ℝ) :
    t * cosh w - (1 + t ^ 2) / 2 = (t - exp (-w)) * (1 - t * exp (-w)) / (2 * exp (-w)) := by
  rw [cosh_eq, exp_neg]
  field_simp
  ring

lemma lam_pos {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) : 0 < lam q t := by
  unfold lam
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  have hl : log q < 0 := log_neg hq hq1
  have h1 : 0 < t - q := by linarith
  have h2 : 0 < 1 - t * q := by nlinarith
  apply div_pos _ (mul_pos h1 h2)
  nlinarith

/-- The calibration for `(α, β, κ) = λ(t², 1, t)` holds iff `λ = λ_{q,t}`, `q = e^{−w}`. -/
theorem calib_iff {w t l : ℝ} (hw : 0 < w) (hqt : exp (-w) < t) (ht : t ≤ 1) :
    (l * t) * cosh w - w = (l * t ^ 2 + l * 1) / 2 ↔ l = lam (exp (-w)) t := by
  have hq : 0 < exp (-w) := exp_pos _
  have hq1 : exp (-w) < 1 := exp_lt_one_iff.mpr (by linarith)
  have h1 : 0 < t - exp (-w) := by linarith
  have h2 : 0 < 1 - t * exp (-w) := by nlinarith
  set D := (t - exp (-w)) * (1 - t * exp (-w)) / (2 * exp (-w)) with hDdef
  have hD : 0 < D := div_pos (mul_pos h1 h2) (by positivity)
  have key : (l * t) * cosh w - w - (l * t ^ 2 + l * 1) / 2 = l * D - w := by
    rw [hDdef, ← calib_factor w t]; ring
  have hlam : lam (exp (-w)) t = w / D := by
    unfold lam; rw [log_exp, hDdef]; field_simp
  rw [hlam, eq_div_iff hD.ne']
  constructor <;> intro h <;> linarith

lemma EE_formula {q : ℝ} (hq : 0 < q) (hq1 : q < 1) :
    EE q = -log (1 - q ^ 2) - 2 * (q ^ 2 / (1 - q ^ 2)) * log q := by
  have hd : 0 < 1 - q ^ 2 := by nlinarith
  unfold EE gfun
  have e1 : q ^ 2 / (1 - q ^ 2) + 1 = (1 - q ^ 2)⁻¹ := by field_simp; ring
  rw [e1, log_inv, log_div (pow_pos hq 2).ne' hd.ne', log_pow]
  push_cast
  field_simp
  ring

/-- Equality in the minimum principle on the two-mode squeezed vacuum (scalar part). -/
theorem tmsv_value {κ α β w : ℝ} (hcal : κ * cosh w - w = (α + β) / 2) (hw : 0 < w) :
    EE (exp (-w)) + (α + β) * (exp (-w) ^ 2 / (1 - exp (-w) ^ 2))
      - κ * (2 * exp (-w) / (1 - exp (-w) ^ 2))
      = -log (1 - exp (-w) ^ 2) - κ * exp (-w) := by
  have hq : 0 < exp (-w) := exp_pos _
  have hq1 : exp (-w) < 1 := exp_lt_one_iff.mpr (by linarith)
  have hd : 0 < 1 - exp (-w) ^ 2 := by nlinarith
  have hab : α + β = κ * (exp w + exp (-w)) - 2 * w := by rw [cosh_eq] at hcal; linarith
  have hinv : exp w = (exp (-w))⁻¹ := by rw [exp_neg, inv_inv]
  rw [EE_formula hq hq1, log_exp, hab, hinv]
  generalize exp (-w) = q at hq hq1 hd ⊢
  field_simp
  ring

/-- `E(q) + λ_{q,t} d_t(q) = −ln(1−q²) − λ_{q,t} t q + λ_{q,t}`. -/
theorem energy_identity {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    EE q + lam q t * dt t q = -log (1 - q ^ 2) - lam q t * t * q + lam q t := by
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  have hd : 0 < 1 - q ^ 2 := by nlinarith
  have h1 : 0 < t - q := by linarith
```

```
  have h2 : 0 < 1 - t * q := by nlinarith
  have h2' : 1 - q * t ≠ 0 := by rw [mul_comm]; exact h2.ne'
  rw [EE_formula hq hq1]
  unfold lam dt
  field_simp
  ring

/-- **Theorem 2 from the minimum principle.** For `m` pairs with `0 < q_j < t_j ≤ 1`,
`λ_j = λ_{q_j,t_j}`, `κ_j = λ_j t_j`, and `⟨T_j⟩ = λ_j(⟨D̂^{(j)}⟩ − 1)`, the minimum principle
`S + ∑ ⟨T_j⟩ ≥ ∑ [−ln(1−q_j²) − κ_j q_j]` implies
`S + ∑ λ_j [⟨D̂^{(j)}⟩ − d_{t_j}(q_j)] ≥ ∑ E(q_j)`. -/
theorem thm2_of_minprinciple {m : ℕ} (q t D : Fin m → ℝ) (Sent : ℝ)
    (hq : ∀ j, 0 < q j) (hqt : ∀ j, q j < t j) (ht : ∀ j, t j ≤ 1)
    (hmin : Sent + ∑ j, lam (q j) (t j) * (D j - 1)
      ≥ ∑ j, (-log (1 - q j ^ 2) - (lam (q j) (t j) * t j) * q j)) :
    Sent + ∑ j, lam (q j) (t j) * (D j - dt (t j) (q j)) ≥ ∑ j, EE (q j) := by
  have hE : ∑ j, EE (q j) = ∑ j, ((-log (1 - q j ^ 2) - (lam (q j) (t j) * t j) * q j)
      + lam (q j) (t j) - lam (q j) (t j) * dt (t j) (q j)) := by
    refine Finset.sum_congr rfl (fun j _ => ?_)
    have := energy_identity (hq j) (hqt j) (ht j)
    linarith
  have hD : ∑ j, lam (q j) (t j) * (D j - dt (t j) (q j))
      = ∑ j, (lam (q j) (t j) * (D j - 1) + lam (q j) (t j)
        - lam (q j) (t j) * dt (t j) (q j)) := by
    refine Finset.sum_congr rfl (fun j _ => ?_); ring
  rw [hE, hD]
  simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib] at hmin ⊢
  linarith

/-- The parameters of Theorem 2 satisfy the hypotheses of Lemma S5: for `0 < q < t ≤ 1`,
`w = −ln q`, `λ = λ_{q,t}`, the triple `(α, β, κ) = λ(t², 1, t)` (and the swapped one
`λ(1, t², t)`) satisfies `Hyp`, with `κ² = αβ`. -/
theorem hyp_of_lam {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) :
    Hyp (lam q t * t) (lam q t * t ^ 2) (lam q t * 1) (-log q) ∧
    Hyp (lam q t * t) (lam q t * 1) (lam q t * t ^ 2) (-log q) := by
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  have hl := lam_pos hq hqt ht
  have ht0 : 0 < t := hq.trans hqt
  have hw : 0 < -log q := neg_pos.mpr (log_neg hq hq1)
  have hexp : exp (-(-log q)) = q := by rw [neg_neg, exp_log hq]
  have hcal := (calib_iff (w := -log q) (t := t) (l := lam q t) hw (by rw [hexp]; exact hqt) ht).mpr
    (by rw [hexp])
  refine ⟨⟨by positivity, by positivity, by positivity, hw, by nlinarith, by linarith⟩,
    ⟨by positivity, by positivity, by positivity, hw, by nlinarith, by linarith⟩⟩

end Thm2Cert
```

Thm2Cert/Comparison.lean

```
/-
Certificate for the comparison `M(x) ≥ γ K(x)` of Lemma S5, with the EXPLICIT constant
    γ = κ sinh w − w coth w − |α − β| / 2  > 0,
and for the rigidity inequality of the Supplemental Material:
    Q(O₁ψ) + Q(O₂ψ) ≤ −γ ∑_{l,r} c_l c_r z†_{lr} K(x_{lr}) z_{lr}   (= −γ (‖O₁ψ‖² + ‖O₂ψ‖²)),
for every finite Schmidt spectrum (kernel directions included), and its many-pair version.
Here Q(O_iψ) are the expressions (S:Q12); their derivation from the Euler equation, and the
identity ∑ c_l c_r z† K z = ‖O₁ψ‖² + ‖O₂ψ‖² (S:defects), are certified in `Derivation.lean`.
-/
import Thm2Cert.Identity

open Real Finset

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

/-! ## `f` is even and nonincreasing in `|x|` -/

lemma f_anti {a c : ℝ} (ha : 0 ≤ a) (hac : a ≤ c) : f c ≤ f a := by
  rcases hac.eq_or_lt with h | h
  · subst h; exact le_rfl
  rcases ha.eq_or_lt with h0 | h0
  · subst h0
    rw [f_zero]
    have h1 := f_mul_sinh c
    have h2 : c < sinh c := self_lt_sinh_iff.mpr h
    have h3 : 0 < sinh c := sinh_pos_iff.mpr h
    nlinarith [f_pos c]
  · have hc : 0 < c := h0.trans h
    have hL := L1 h0 h
    have e1 := f_mul_sinh a
    have e2 := f_mul_sinh c
    have hsa : 0 < sinh a := sinh_pos_iff.mpr h0
    have hsc : 0 < sinh c := sinh_pos_iff.mpr hc
    have key : (f c - f a) * (sinh a * sinh c) = c * sinh a - a * sinh c := by
      linear_combination sinh a * e2 - sinh c * e1
    have : (f c - f a) * (sinh a * sinh c) < 0 := by rw [key]; linarith
```

```
    nlinarith [mul_pos hsa hsc]

lemma f_abs (x : ℝ) : f |x| = f x := by
  rcases abs_choice x with h | h
  · rw [h]
  · rw [h, f_neg]

/-! ## Two more half-angle identities -/

lemma minus_identity (κ0 w S C sy cy sxh cxh cx sxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (hsxw : sxw = sp * C + cp * S) :
    S * ((κ0 * S * cy + w * sy + C * F) - (κ0 * S + F * cx))
      = 2 * w * sp ^ 2 + 2 * sp * (sxw * (w - S * F)) := by
  subst hw; subst hcx; subst hsxw; subst hS; subst hC hsy hcy hsxh hcxh
  linear_combination (-2 * cr * sr * (F * cr ^ 2 + F * sr ^ 2 - 2 * cr * sr * κ0)) * hp
    + (-2 * sp ^ 2 * κ0 * (cr ^ 2 - sr ^ 2 + 1)) * hr

lemma plus_identity (κ0 w S C sy cy sxh cxh cx cxw sp cp sr cr F : ℝ)
    (hp : cp ^ 2 - sp ^ 2 = 1) (hr : cr ^ 2 - sr ^ 2 = 1)
    (hw : w = κ0 * (C - 1)) (hC : C = cr ^ 2 + sr ^ 2) (hS : S = 2 * sr * cr)
    (hsy : sy = 2 * sp * cp) (hcy : cy = cp ^ 2 + sp ^ 2)
    (hsxh : sxh = sp * cr + cp * sr) (hcxh : cxh = cp * cr + sp * sr)
    (hcx : cx = cxh ^ 2 + sxh ^ 2) (hcxw : cxw = cp * C + sp * S) :
    S * ((κ0 * S * cy + w * sy + C * F) + (κ0 * S + F * cx))
      = 2 * w * cp ^ 2 + 2 * w * cp * cxw + S * F * (C + cx) := by
  subst hw; subst hcx; subst hcxw; subst hS; subst hC hsy hcy hsxh hcxh
  linear_combination (-2 * κ0 * (cr ^ 4 + sr ^ 4 - 1)) * hp
    + (-2 * κ0 * (sp ^ 2 + 1) * (cr ^ 2 - sr ^ 2 + 1)) * hr

/-! ## Eigenvalue bounds at `κ = κ₀`, `δ = 0`: `N₀ ≥ (w / sinh w) K̃` -/

/-- Both eigenvalues of `M/(2q)` at `κ = κ₀`, `δ = 0` dominate `w / sinh w` times those of
`K/(2q) = [[cosh(x−w), −1], [−1, cosh(x−w)]]`. -/
lemma eig_bounds {w : ℝ} (hw : 0 < w) (κ0 : ℝ) (h3 : κ0 * (cosh w - 1) = w) (x : ℝ) :
    w * (cosh (x - w) - 1) ≤ sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w)
        + cosh w * f x) - (κ0 * sinh w + f x * cosh x)) ∧
    w * (cosh (x - w) + 1) ≤ sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w)
        + cosh w * f x) + (κ0 * sinh w + f x * cosh x)) := by
  have hp : cosh ((x - w) / 2) ^ 2 - sinh ((x - w) / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hr : cosh (w / 2) ^ 2 - sinh (w / 2) ^ 2 = 1 := by rw [cosh_sq]; ring
  have hCr : cosh w = cosh (w / 2) ^ 2 + sinh (w / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hSr : sinh w = 2 * sinh (w / 2) * cosh (w / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hsy : sinh (x - w) = 2 * sinh ((x - w) / 2) * cosh ((x - w) / 2) := by
    rw [← sinh_two_mul]; congr 1; ring
  have hcy : cosh (x - w) = cosh ((x - w) / 2) ^ 2 + sinh ((x - w) / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hsxh : sinh (x / 2) = sinh ((x - w) / 2) * cosh (w / 2)
      + cosh ((x - w) / 2) * sinh (w / 2) := by
    rw [← sinh_add]; congr 1; ring
  have hcxh : cosh (x / 2) = cosh ((x - w) / 2) * cosh (w / 2)
      + sinh ((x - w) / 2) * sinh (w / 2) := by
    rw [← cosh_add]; congr 1; ring
  have hcx : cosh x = cosh (x / 2) ^ 2 + sinh (x / 2) ^ 2 := by
    rw [← cosh_two_mul]; congr 1; ring
  have hsxw : sinh ((x + w) / 2) = sinh ((x - w) / 2) * cosh w
      + cosh ((x - w) / 2) * sinh w := by
    rw [← sinh_add]; congr 1; ring
  have hcxw : cosh ((x + w) / 2) = cosh ((x - w) / 2) * cosh w
      + sinh ((x - w) / 2) * sinh w := by
    rw [← cosh_add]; congr 1; ring
  have hm := minus_identity κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (sinh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hsxw
  have hpl := plus_identity κ0 w (sinh w) (cosh w) (sinh (x - w)) (cosh (x - w)) (sinh (x / 2))
    (cosh (x / 2)) (cosh x) (cosh ((x + w) / 2)) (sinh ((x - w) / 2)) (cosh ((x - w) / 2))
    (sinh (w / 2)) (cosh (w / 2)) (f x) hp hr h3.symm hCr hSr hsy hcy hsxh hcxh hcx hcxw
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hfw : f w * sinh w = w := f_mul_sinh w
  -- sign: sinh((x−w)/2) · sinh((x+w)/2) · (w − sinh w · f x) ≥ 0
  have hsign : 0 ≤ sinh ((x - w) / 2) * (sinh ((x + w) / 2) * (w - sinh w * f x)) := by
    rcases le_total w x with hxw | hxw
    · -- x ≥ w
      have h1 : 0 ≤ sinh ((x - w) / 2) := sinh_nonneg_iff.mpr (by linarith)
      have h2 : 0 ≤ sinh ((x + w) / 2) := sinh_nonneg_iff.mpr (by linarith)
      have h3' : f x ≤ f w := by
        rw [← f_abs x]; exact f_anti hw.le (le_trans hxw (le_abs_self x))
      have h4 : 0 ≤ w - sinh w * f x := by nlinarith
      exact mul_nonneg h1 (mul_nonneg h2 h4)
    · have h1 : sinh ((x - w) / 2) ≤ 0 := sinh_nonpos_iff.mpr (by linarith)
      rcases le_total (-w) x with hxw' | hxw'
      · -- −w ≤ x ≤ w
        have h2 : 0 ≤ sinh ((x + w) / 2) := sinh_nonneg_iff.mpr (by linarith)
        have h3' : f w ≤ f x := by
          rw [← f_abs x]; exact f_anti (abs_nonneg x) (abs_le.mpr (hxw', hxw))
        have h4 : w - sinh w * f x ≤ 0 := by nlinarith
```

```
        have : sinh ((x + w) / 2) * (w - sinh w * f x) ≤ 0 :=
          mul_nonpos_of_nonneg_of_nonpos h2 h4
        exact mul_nonneg_of_nonpos_of_nonpos h1 this
      · -- x ≤ −w
        have h2 : sinh ((x + w) / 2) ≤ 0 := sinh_nonpos_iff.mpr (by linarith)
        have h3' : f x ≤ f w := by
          rw [← f_abs x]; exact f_anti hw.le (by rw [abs_of_nonpos (by linarith)]; linarith)
        have h4 : 0 ≤ w - sinh w * f x := by nlinarith
        have : sinh ((x + w) / 2) * (w - sinh w * f x) ≤ 0 :=
          mul_nonpos_of_nonpos_of_nonneg h2 h4
        exact mul_nonneg_of_nonpos_of_nonpos h1 this
  have hcxw_pos : 0 < cosh ((x + w) / 2) := cosh_pos _
  have hcp : 0 < cosh ((x - w) / 2) := cosh_pos _
  have hCx : 0 < cosh w + cosh x := by linarith [cosh_pos w, cosh_pos x]
  have hw2 : w * (cosh ((x - w) / 2) ^ 2 - sinh ((x - w) / 2) ^ 2) = w := by rw [hp, mul_one]
  constructor
  · rw [hm, hcy]; linarith
  · rw [hpl, hcy]
    have : 0 ≤ sinh w * f x * (cosh w + cosh x) :=
      (mul_pos (mul_pos hS (f_pos x)) hCx).le
    have : 0 ≤ 2 * w * cosh ((x - w) / 2) * cosh ((x + w) / 2) :=
      (mul_pos (mul_pos (by linarith) hcp) hcxw_pos).le
    linarith

/-! ## The explicit constant `γ` -/

/-- `γ = κ sinh w − w coth w − |α − β|/2`. -/
def gam (κ α β w : ℝ) : ℝ := κ * sinh w - w * cosh w / sinh w - |α - β| / 2

variable {κ α β w : ℝ}

lemma gam_mul_sinh (hw : 0 < w) :
    gam κ α β w * sinh w = κ * sinh w ^ 2 - w * cosh w - |α - β| / 2 * sinh w := by
  have hS : sinh w ≠ 0 := (sinh_pos_iff.mpr hw).ne'
  unfold gam
  rw [sub_mul, sub_mul, div_mul_cancel₀ _ hS]; ring

theorem gam_pos (h : Hyp κ α β w) : 0 < gam κ α β w := by
  have hw := h.hw
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr hw.ne'; linarith
  have hSC : sinh w ^ 2 = cosh w ^ 2 - 1 := by rw [cosh_sq]; ring
  have hk := kappa0_le h
  rw [div_le_iff₀ hC1] at hk
  -- R := κ S² − w C > 0
  have hR : 0 < κ * sinh w ^ 2 - w * cosh w := by
    have : κ * sinh w ^ 2 = κ * (cosh w - 1) * (cosh w + 1) := by rw [hSC]; ring
    nlinarith [cosh_pos w]
  -- (|δ| S / 2)² < R²
  have hδ : (α - β) ^ 2 ≤ 4 * ((κ * cosh w - w) ^ 2 - κ ^ 2) := by
    have hs : α + β = 2 * (κ * cosh w - w) := by linarith [h.hcal]
    have e : (α - β) ^ 2 = (α + β) ^ 2 - 4 * (α * β) := by ring
    rw [e, hs]; nlinarith [h.hκαβ]
  have hsq : (|α - β| / 2 * sinh w) ^ 2 < (κ * sinh w ^ 2 - w * cosh w) ^ 2 := by
    have e : (κ * sinh w ^ 2 - w * cosh w) ^ 2
        = ((κ * cosh w - w) ^ 2 - κ ^ 2) * sinh w ^ 2 + w ^ 2 := by
      rw [hSC]; ring
    rw [e, mul_pow, div_pow, sq_abs]
    have : 0 < w ^ 2 := by positivity
    have := mul_le_mul_of_nonneg_right hδ (sq_nonneg (sinh w))
    nlinarith
  have hlt := abs_lt_of_sq_lt_sq hsq hR.le
  have hlt' : |α - β| / 2 * sinh w < κ * sinh w ^ 2 - w * cosh w := lt_of_abs_lt hlt
  have hpos : 0 < gam κ α β w * sinh w := by rw [gam_mul_sinh hw]; linarith
  exact (mul_pos_iff_of_pos_right hS).mp hpos

/-- `γ ≤ α − qκ` and `q γ ≤ κ − q α` (the boundary weights dominate `γ`). -/
lemma gam_le (h : Hyp κ α β w) :
    gam κ α β w ≤ α - exp (-w) * κ ∧ exp (-w) * gam κ α β w ≤ κ - exp (-w) * α := by
  have hw := h.hw
  have hS : 0 < sinh w := sinh_pos_iff.mpr hw
  have hq : cosh w - sinh w = exp (-w) := cosh_sub_sinh w
  have hE : exp (-w) * (cosh w + sinh w) = 1 := by
    rw [cosh_add_sinh, ← exp_add]; simp
  have hwC : w ≤ w * cosh w / sinh w := by
    rw [le_div_iff₀ hS]; nlinarith [sinh_lt_cosh w]
  have hwC0 : 0 ≤ w * cosh w / sinh w := by linarith
  have hab := h.hcal
  have hd1 : -(α - β) ≤ |α - β| := neg_le_abs _
  have hd2 : α - β ≤ |α - β| := le_abs_self _
  have hqk : κ * cosh w - κ * sinh w = exp (-w) * κ := by rw [← mul_sub, hq, mul_comm]
  unfold gam
  constructor
  · linarith
  · have e : κ - exp (-w) * α - exp (-w) * (κ * sinh w - w * cosh w / sinh w - |α - β| / 2)
        = exp (-w) * (κ * (cosh w + sinh w) - α
          - (κ * sinh w - w * cosh w / sinh w - |α - β| / 2)) := by
      linear_combination (-κ) * hE
    have : 0 ≤ κ * (cosh w + sinh w) - α - (κ * sinh w - w * cosh w / sinh w - |α - β| / 2) := by
      linarith
    nlinarith [exp_pos (-w)]
```

```
223
224 lemma gam_swap : gam κ α β w = gam κ β α w := by unfold gam; rw [abs_sub_comm]
225
226 /-! ## `M(x)/(2q) ≥ γ K(x)/(2q)` -/
227
228 lemma amgm_cross (δ sy cy s d : ℝ) (hy : cy ^ 2 - sy ^ 2 = 1) (hc : 1 ≤ cy) :
229     2 * δ * sy * s * d ≤ |δ| * ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) := by
230   have hX : 0 ≤ (cy - 1) * s ^ 2 + (cy + 1) * d ^ 2 := by
231     have := sq_nonneg s; have := sq_nonneg d; nlinarith
232   have hsq : (2 * sy * s * d) ^ 2 ≤ ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) ^ 2 := by
233     have : ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) ^ 2 - (2 * sy * s * d) ^ 2
234         = ((cy - 1) * s ^ 2 - (cy + 1) * d ^ 2) ^ 2 + 4 * s ^ 2 * d ^ 2 * (cy ^ 2 - sy ^ 2 - 1) := by
235       ring
236     rw [hy, sub_self, mul_zero, add_zero] at this
237     nlinarith [sq_nonneg ((cy - 1) * s ^ 2 - (cy + 1) * d ^ 2)]
238   have habs := abs_le_of_sq_le_sq hsq hX
239   calc 2 * δ * sy * s * d = δ * (2 * sy * s * d) := by ring
240     _ ≤ |δ * (2 * sy * s * d)| := le_abs_self _
241     _ = |δ| * |2 * sy * s * d| := abs_mul _ _
242     _ ≤ |δ| * ((cy - 1) * s ^ 2 + (cy + 1) * d ^ 2) :=
243         mul_le_mul_of_nonneg_left habs (abs_nonneg _)
244
245 theorem N_comparison (h : Hyp κ α β w) (x u v : ℝ) :
246     gam κ α β w * (cosh (x - w) * u ^ 2 - 2 * u * v + cosh (x - w) * v ^ 2)
247       ≤ mdiag κ α w x * u ^ 2 - 2 * moff κ w x * u * v + mdiag κ β w x * v ^ 2 := by
248   have hw := h.hw
249   have hS : 0 < sinh w := sinh_pos_iff.mpr hw
250   have hC1 : 0 < cosh w - 1 := by have := one_lt_cosh.mpr hw.ne'; linarith
251   set κ0 := w / (cosh w - 1) with hκ0def
252   have h3 : κ0 * (cosh w - 1) = w := div_mul_cancel₀ w hC1.ne'
253   have hSC : sinh w ^ 2 = cosh w ^ 2 - 1 := by rw [cosh_sq]; ring
254   obtain (hAm, hAp) := eig_bounds hw κ0 h3 x
255   have hg := gam_mul_sinh (κ := κ) (α := α) (β := β) hw
256   -- Γ S = γ S + |δ| S / 2, with Γ = w/S + (κ − κ₀) S
257   have hΓ : w + (κ - κ0) * sinh w ^ 2 = gam κ α β w * sinh w + |α - β| / 2 * sinh w := by
258     rw [hg, hSC]; linear_combination (-(cosh w + 1)) * h3
259   have hy : cosh (x - w) ^ 2 - sinh (x - w) ^ 2 = 1 := by rw [cosh_sq]; ring
260   have hcy1 : 1 ≤ cosh (x - w) := one_le_cosh _
261   -- P₁ = A − c − γ (cosh y − 1) ≥ |δ|/2 (cosh y − 1), P₂ likewise
262   set A := κ * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x with hA
263   set c := κ * sinh w + f x * cosh x with hc
264   have hP1 : sinh w * (|α - β| / 2 * (cosh (x - w) - 1))
265       ≤ sinh w * (A - c - gam κ α β w * (cosh (x - w) - 1)) := by
266     have e : sinh w * (A - c - gam κ α β w * (cosh (x - w) - 1))
267         = sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
268           - (κ0 * sinh w + f x * cosh x)) - w * (cosh (x - w) - 1)
269           + (cosh (x - w) - 1) * (w + (κ - κ0) * sinh w ^ 2 - gam κ α β w * sinh w) := by
270       rw [hA, hc]; ring
271     rw [e, hΓ]; nlinarith
272   have hP2 : sinh w * (|α - β| / 2 * (cosh (x - w) + 1))
273       ≤ sinh w * (A + c - gam κ α β w * (cosh (x - w) + 1)) := by
274     have e : sinh w * (A + c - gam κ α β w * (cosh (x - w) + 1))
275         = sinh w * ((κ0 * sinh w * cosh (x - w) + w * sinh (x - w) + cosh w * f x)
276           + (κ0 * sinh w + f x * cosh x)) - w * (cosh (x - w) + 1)
277           + (cosh (x - w) + 1) * (w + (κ - κ0) * sinh w ^ 2 - gam κ α β w * sinh w) := by
278       rw [hA, hc]; ring
279     rw [e, hΓ]; nlinarith
280   have hP1' := le_of_mul_le_mul_left hP1 hS
281   have hP2' := le_of_mul_le_mul_left hP2 hS
282   -- diagonal entries through A, using the calibration
283   have hmα : mdiag κ α w x = A - (α - β) / 2 * sinh (x - w) := by
284     unfold mdiag; rw [hA]; linear_combination sinh (x - w) * h.hcal
285   have hmβ : mdiag κ β w x = A + (α - β) / 2 * sinh (x - w) := by
286     unfold mdiag; rw [hA]; linear_combination sinh (x - w) * h.hcal
287   have hmo : moff κ w x = c := by unfold moff b; rw [hc]
288   rw [hmα, hmβ, hmo]
289   -- change of variables s = (u+v)/2, d = (u−v)/2
290   have e : (A - (α - β) / 2 * sinh (x - w)) * u ^ 2 - 2 * c * u * v
291       + (A + (α - β) / 2 * sinh (x - w)) * v ^ 2
292       - gam κ α β w * (cosh (x - w) * u ^ 2 - 2 * u * v + cosh (x - w) * v ^ 2)
293       = 2 * (A - c - gam κ α β w * (cosh (x - w) - 1)) * ((u + v) / 2) ^ 2
294         - 2 * (α - β) * sinh (x - w) * ((u + v) / 2) * ((u - v) / 2)
295         + 2 * (A + c - gam κ α β w * (cosh (x - w) + 1)) * ((u - v) / 2) ^ 2 := by ring
296   have hcross := amgm_cross (α - β) (sinh (x - w)) (cosh (x - w)) ((u + v) / 2) ((u - v) / 2)
297     hy hcy1
298   have i1 := mul_le_mul_of_nonneg_right hP1' (sq_nonneg ((u + v) / 2))
299   have i2 := mul_le_mul_of_nonneg_right hP2' (sq_nonneg ((u - v) / 2))
300   have : 0 ≤ (A - (α - β) / 2 * sinh (x - w)) * u ^ 2 - 2 * c * u * v
301       + (A + (α - β) / 2 * sinh (x - w)) * v ^ 2
302       - gam κ α β w * (cosh (x - w) * u ^ 2 - 2 * u * v + cosh (x - w) * v ^ 2) := by
303     rw [e]; linarith
304   linarith
305
306 /-! ## Lemma S5, comparison part: `M(x) ≥ γ K(x)` -/
307
308 /-- Diagonal entry of `K(x)`: `e^{−x} + q² eˣ` (the off-diagonal entry is `−2q`). -/
309 def Kdiag (q x : ℝ) : ℝ := exp (-x) + q ^ 2 * exp x
310
311 lemma Kdiag_eq (w x : ℝ) : Kdiag (exp (-w)) x = 2 * exp (-w) * cosh (x - w) := by
312   unfold Kdiag
313   simp only [cosh_eq, Real.exp_neg, Real.exp_sub, neg_sub]
```

```
  field_simp
  ring

/-- The quadratic form of `K(x)`. -/
def QK (w x u v : ℝ) : ℝ :=
  Kdiag (exp (-w)) x * u ^ 2 + 2 * (-2 * exp (-w)) * u * v + Kdiag (exp (-w)) x * v ^ 2

/-- **Lemma S5 (iv)**: `M(x) ≥ γ K(x)` for every real `x`, with the explicit
`γ = κ sinh w − w coth w − |α−β|/2 > 0` (`gam_pos`). -/
theorem lemmaS5_gamma (h : Hyp κ α β w) (x u v : ℝ) :
    gam κ α β w * QK w x u v ≤ QM κ α β w x u v := by
  rw [QM_eq]
  unfold QK
  rw [Kdiag_eq]
  have := N_comparison h x u v
  have hq : 0 < 2 * exp (-w) := by positivity
  have := mul_le_mul_of_nonneg_left this hq.le
  linarith

/-- A real symmetric 2×2 form that is nonnegative on `ℝ²` is nonnegative on `z = (A, conj B)`. -/
lemma psd_complex {p11 p12 p22 : ℝ}
    (hP : ∀ u v : ℝ, 0 ≤ p11 * u ^ 2 + 2 * p12 * u * v + p22 * v ^ 2) (A B : ℂ) :
    0 ≤ p11 * Complex.normSq A + p22 * Complex.normSq B + 2 * p12 * (A * B).re := by
  have hre1 : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  have hre2 : -(‖A‖ * ‖B‖) ≤ (A * B).re := by
    have := Complex.re_le_norm (-(A * B))
    rw [norm_neg, Complex.norm_mul, Complex.neg_re] at this
    linarith
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  rcases le_total 0 p12 with hp | hp
  · have := hP ‖A‖ (-‖B‖)
    nlinarith [mul_le_mul_of_nonneg_left hre2 hp]
  · have := hP ‖A‖ ‖B‖
    nlinarith [mul_le_mul_of_nonpos_left hre1 hp]

/-- Complex form of `M(x) ≥ γ K(x)`, for `z = (A, conj B)`. -/
theorem lemmaS5_gamma_complex (h : Hyp κ α β w) (x : ℝ) (A B : ℂ) :
    gam κ α β w * (Kdiag (exp (-w)) x * (Complex.normSq A + Complex.normSq B)
        + 2 * (-2 * exp (-w)) * (A * B).re)
      ≤ Mdiag κ α (exp (-w)) x * Complex.normSq A + Mdiag κ β (exp (-w)) x * Complex.normSq B
        + 2 * Moff κ (exp (-w)) x * (A * B).re := by
  have hP : ∀ u v : ℝ, 0 ≤ (Mdiag κ α (exp (-w)) x - gam κ α β w * Kdiag (exp (-w)) x) * u ^ 2
      + 2 * (Moff κ (exp (-w)) x - gam κ α β w * (-2 * exp (-w))) * u * v
      + (Mdiag κ β (exp (-w)) x - gam κ α β w * Kdiag (exp (-w)) x) * v ^ 2 := by
    intro u v
    have := lemmaS5_gamma h x u v
    unfold QM QK at this
    linarith
  have := psd_complex hP A B
  linarith

/-! ## Rigidity: `Q(O₁ψ) + Q(O₂ψ) ≤ −γ (‖O₁ψ‖² + ‖O₂ψ‖²)` -/

/-- Summand of `‖O₁ψ‖² + ‖O₂ψ‖² = ∑ c_l c_r z† K(x_lr) z` (Eq. S:defects) in the
`p`-variables: `c_l c_r K₁₁ = p_r + q² p_l`, `c_l c_r K₁₂ = −2q c_l c_r`. -/
def kterm (q cl cr : ℝ) (A B : ℂ) : ℝ :=
  (cr ^ 2 + q ^ 2 * cl ^ 2) * (Complex.normSq A + Complex.normSq B)
    - 4 * q * (cl * cr) * (A * B).re

lemma kterm_nonneg {q cl cr : ℝ} (hq : 0 ≤ q) (hl : 0 ≤ cl) (hr : 0 ≤ cr) (A B : ℂ) :
    0 ≤ kterm q cl cr A B := by
  unfold kterm
  have hre : (A * B).re ≤ ‖A‖ * ‖B‖ := by
    rw [← Complex.norm_mul]; exact Complex.re_le_norm _
  have hm : 0 ≤ q * (cl * cr) := mul_nonneg hq (mul_nonneg hl hr)
  have hP : 2 * (q * (cl * cr)) ≤ cr ^ 2 + q ^ 2 * cl ^ 2 := by nlinarith [sq_nonneg (cr - q * cl)]
  have hN : 2 * (‖A‖ * ‖B‖) ≤ ‖A‖ ^ 2 + ‖B‖ ^ 2 := by nlinarith [sq_nonneg (‖A‖ - ‖B‖)]
  have hab : 0 ≤ ‖A‖ * ‖B‖ := by positivity
  rw [Complex.normSq_eq_norm_sq, Complex.normSq_eq_norm_sq]
  have i1 := mul_le_mul_of_nonneg_left hre hm
  have i2 := mul_le_mul_of_nonneg_right hP hab
  have i3 := mul_le_mul_of_nonneg_left hN (by positivity : (0:ℝ) ≤ cr ^ 2 + q ^ 2 * cl ^ 2)
  nlinarith

/-- Termwise comparison, for all Schmidt coefficients `cl, cr ≥ 0`. -/
theorem term_ge (h : Hyp κ α β w) {cl cr : ℝ} (hl : 0 ≤ cl) (hr : 0 ≤ cr) (A B : ℂ) :
    gam κ α β w * kterm (exp (-w)) cl cr A B ≤ term κ α β (exp (-w)) cl cr A B := by
  obtain (ga1, ga2) := gam_le h
  obtain (gb1, gb2) := gam_le h.swap
  rw [← gam_swap] at gb1 gb2
  have he : 0 < exp (-w) := exp_pos _
  have nA := Complex.normSq_nonneg A
  have nB := Complex.normSq_nonneg B
  rcases hl.eq_or_lt with hl0 | hlp
  · subst hl0
    unfold term wdiag woff kterm
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, add_zero, sub_zero]
    have i1 := mul_le_mul_of_nonneg_right ga1 (mul_nonneg (sq_nonneg cr) nA)
    have i2 := mul_le_mul_of_nonneg_right gb1 (mul_nonneg (sq_nonneg cr) nB)
```

```
    nlinarith
  rcases hr.eq_or_lt with hr0 | hrp
  · subst hr0
    unfold term wdiag woff kterm
    simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, zero_mul, mul_zero,
      Real.sqrt_zero, add_zero, sub_zero, zero_add]
    have i1 := mul_le_mul_of_nonneg_right ga2 (mul_nonneg (mul_nonneg he.le (sq_nonneg cl)) nA)
    have i2 := mul_le_mul_of_nonneg_right gb2 (mul_nonneg (mul_nonneg he.le (sq_nonneg cl)) nB)
    nlinarith
  · have e1 := E1 hlp hrp
    have e2 := E2 hlp hrp
    have e3 := E3 hlp hrp
    have e4 := E4 hlp hrp
    have keyM : term κ α β (exp (-w)) cl cr A B = cl * cr *
        (Mdiag κ α (exp (-w)) (log (cl / cr)) * Complex.normSq A
          + Mdiag κ β (exp (-w)) (log (cl / cr)) * Complex.normSq B
          + 2 * Moff κ (exp (-w)) (log (cl / cr)) * (A * B).re) := by
      unfold term wdiag woff Mdiag Moff
      linear_combination
        (-exp (-w) * (κ - exp (-w) * α) * Complex.normSq A
          - exp (-w) * (κ - exp (-w) * β) * Complex.normSq B) * e1
        + (-(α - exp (-w) * κ) * Complex.normSq A - (β - exp (-w) * κ) * Complex.normSq B) * e2
        + (1 + exp (-w) ^ 2) * (Complex.normSq A + Complex.normSq B) * e3
        - 2 * exp (-w) * (A * B).re * e4
    have keyK : kterm (exp (-w)) cl cr A B = cl * cr *
        (Kdiag (exp (-w)) (log (cl / cr)) * (Complex.normSq A + Complex.normSq B)
          + 2 * (-2 * exp (-w)) * (A * B).re) := by
      unfold kterm Kdiag
      linear_combination
        (-(Complex.normSq A + Complex.normSq B)) * e2
        + (-exp (-w) ^ 2 * (Complex.normSq A + Complex.normSq B)) * e1
    rw [keyM, keyK]
    have := mul_le_mul_of_nonneg_left (lemmaS5_gamma_complex h (log (cl / cr)) A B)
      (mul_pos hlp hrp).le
    linarith

variable {ι : Type*} [Fintype ι]

/-- **Rigidity inequality** for one pair: `Q(O₁ψ) + Q(O₂ψ) ≤ −γ ∑ c_l c_r z† K z`. -/
theorem rigidity (h : Hyp κ α β w) (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l)
    (A B : ι → ι → ℂ) (nA nB : ℝ)
    (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
    (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
    (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r))
    (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
    Q1 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      + Q2 α β κ (exp (-w)) nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B)
      ≤ -(gam κ α β w * ∑ l, ∑ r, kterm (exp (-w)) (c l) (c r) (A l r) (B l r)) := by
  rw [mainid_general c A B α β κ (exp (-w)) nA nB hA0 hA1 hB0 hB1, neg_le_neg_iff,
    Finset.mul_sum]
  refine Finset.sum_le_sum (fun l _ => ?_)
  rw [Finset.mul_sum]
  exact Finset.sum_le_sum (fun r _ => term_ge h (hc l) (hc r) (A l r) (B l r))

/-- **Rigidity, many pairs.** For `m` pairs `j` (all sharing the Schmidt coefficients `c`)
with hypotheses `Hyp κ_j α_j β_j w_j`, and any `0 < γ₀ ≤ min_j γ_j`:
`∑_j [Q(O₁,ⱼψ) + Q(O₂,ⱼψ)] ≤ −γ₀ ∑_j ∑ c_l c_r z†_j K_j z_j`. In particular, if the left side
is `≥ 0` (second-order stability with `(O_{i,j}) = 0`), every defect sum vanishes. -/
theorem rigidity_multi {m : ℕ} (κ α β w : Fin m → ℝ) (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
    (γ0 : ℝ) (hγ0 : 0 < γ0) (hγ : ∀ j, γ0 ≤ gam (κ j) (α j) (β j) (w j))
    (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
    (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
    (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
    (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
    (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r)) :
    (∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))))
      ≤ -(γ0 * ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r)) ∧
    (0 ≤ ∑ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j))) →
      ∀ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) = 0) := by
  have hK : ∀ j, 0 ≤ ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) := fun j =>
    Finset.sum_nonneg (fun l _ => Finset.sum_nonneg (fun r _ =>
      kterm_nonneg (exp_pos _).le (hc l) (hc r) _ _))
  have hj : ∀ j, (Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j))
        (sfA c (A j)) (sfA c (B j)) (sfC c (A j) (B j))
      + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
        (sfA c (B j)) (sfC c (A j) (B j)))
      ≤ -(γ0 * ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r)) := by
    intro j
    have := rigidity (h j) c hc (A j) (B j) (nA j) (nB j) (hA0 j) (hA1 j) (hB0 j) (hB1 j)
    have := mul_le_mul_of_nonneg_right (hγ j) (hK j)
    linarith
  have hsum := Finset.sum_le_sum (fun j (_ : j ∈ Finset.univ) => hj j)
  rw [Finset.sum_neg_distrib, ← Finset.mul_sum] at hsum
  refine (hsum, fun h0 j => ?_)
  have htot : γ0 * ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) ≤ 0 := by
```

```
    linarith
  have hnn : 0 ≤ ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) :=
    Finset.sum_nonneg (fun j _ => hK j)
  have hle : ∑ j, ∑ l, ∑ r, kterm (exp (-w j)) (c l) (c r) (A j l r) (B j l r) ≤ 0 := by
    by_contra hcon
    have := mul_pos hγ0 (not_le.mp hcon)
    linarith
  have hzero := le_antisymm hle hnn
  exact (Finset.sum_eq_zero_iff_of_nonneg (fun j _ => hK j)).mp hzero j (Finset.mem_univ j)

end Thm2Cert
```

Thm2Cert/Derivation.lean

```
/-
Certificate for the derivation of Eqs. (S:Q12) and (S:defects) of the Supplemental Material,
in the Schmidt-basis matrix model:

  * coefficient matrices: ψ ↦ C = diag(c_l); (a ψ)_lm = A_lm c_m, (b† ψ)_lm = c_l conj(B_lm),
    (a† ψ)_lm = conj(A_ml) c_m, (b ψ)_lm = c_l B_ml, with A_lr = ⟨u_l|a|u_r⟩, B_lr = ⟨v_l|b|v_r⟩;
  * test vectors O₁ψ = (a − q b†)ψ, O₂ψ = (q a† − b)ψ;
  * (S:defects): ‖O₁ψ‖² + ‖O₂ψ‖² = ∑ c_l c_r z† K z  (exact, any real c_l);
  * (S:Q12): Q(O_iψ) = Re⟨O_iψ, (T − μ − F) O_iψ⟩ equals the stated expressions, where
    (T − μ) O ψ is replaced by O G(C) + [T, O] ψ (Euler equation (S:euler)), with
    G(C) = diag(c_l ln p_l), [T, a − q b†] = (qκ − α) a + (κ − qβ) b†,
    [T, q a† − b] = (qα − κ) a† + (β − qκ) b, and F the Daleckii–Krein divided difference
    F_lr = (p_l ln p_l − p_r ln p_r)/(p_l − p_r), F_ll = 1 + ln p_l  (c_l ≥ 0, kernel entries by continuity, with 0 ln 0 = 0).
  * `stationary_rigidity`: chaining (S:Q12), (S:mainid), Lemma S5 (M ≥ γK) and (S:defects):
    if the second variations along all test vectors sum to ≥ 0, every O_{i,j}ψ vanishes.

Inputs (not derived here):
  * the matrix model itself: ψ in Schmidt form on a finite index set (support plus the finitely
    many kernel directions reached by a, a†, b, b†), with A_lr, B_lr the matrix elements;
    `O1v`, `O2v` are DEFINED as the resulting coefficient matrices;
  * the sum rules (S:wccr) (consequences of the canonical commutation relations);
  * the Euler equation (S:euler) and the commutators [T, O] (CCR), which enter by writing
    (T − μ)Oψ as OG + [T,O]ψ directly in the statements;
  * the second-variation formula Q(Y) = Re⟨Y, (T − μ − F)Y⟩ (Daleckii–Krein) and the
    stability inequality (S:stability), which enters as the hypothesis `0 ≤ ∑ Q`.
-/
import Thm2Cert.Comparison

open Real Finset ComplexConjugate

set_option autoImplicit false

noncomputable section

namespace Thm2Cert

variable {ι : Type*} [Fintype ι]

/-! ## (S:defects) -/

/-- Coefficient matrix of `O₁ψ = (a − q b†)ψ`. -/
def O1v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  A l m * (c m : ℂ) - (q : ℂ) * (c l : ℂ) * conj (B l m)

/-- Coefficient matrix of `O₂ψ = (q a† − b)ψ`. -/
def O2v (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) (l m : ι) : ℂ :=
  (q : ℂ) * (c m : ℂ) * conj (A m l) - (c l : ℂ) * B m l

lemma defect_term (q cl cr : ℝ) (A B : ℂ) :
    Complex.normSq (A * (cr : ℂ) - (q : ℂ) * (cl : ℂ) * conj B)
      + Complex.normSq ((q : ℂ) * (cl : ℂ) * conj A - (cr : ℂ) * B) = kterm q cl cr A B := by
  unfold kterm
  simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.mul_re,
    Complex.mul_im, Complex.ofReal_re, Complex.ofReal_im, Complex.conj_re, Complex.conj_im]
  ring

/-- **Eq. (S:defects)**: `‖O₁ψ‖² + ‖O₂ψ‖² = ∑_{l,r} c_l c_r z† K(x_lr) z` (in `p`-variables). -/
theorem defects (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ) :
    ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) + ∑ l, ∑ m, Complex.normSq (O2v q c A B l m)
      = ∑ l, ∑ r, kterm q (c l) (c r) (A l r) (B l r) := by
  rw [Finset.sum_comm (f := fun l m => Complex.normSq (O2v q c A B l m)), ← Finset.sum_add_distrib]
  refine Finset.sum_congr rfl (fun l _ => ?_)
  rw [← Finset.sum_add_distrib]
  refine Finset.sum_congr rfl (fun r _ => ?_)
  unfold O1v O2v
  exact defect_term q (c l) (c r) (A l r) (B l r)

/-- If both test vectors have zero norm, all their coefficients vanish. -/
theorem defects_zero (q : ℝ) (c : ι → ℝ) (A B : ι → ι → ℂ)
    (h0 : ∑ l, ∑ r, kterm q (c l) (c r) (A l r) (B l r) = 0) :
    ∀ l m, O1v q c A B l m = 0 ∧ O2v q c A B l m = 0 := by
  rw [← defects] at h0
  have n1 : 0 ≤ ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) :=
    Finset.sum_nonneg (fun _ _ => Finset.sum_nonneg (fun _ _ => Complex.normSq_nonneg _))
  have n2 : 0 ≤ ∑ l, ∑ m, Complex.normSq (O2v q c A B l m) :=
```

```
    Finset.sum_nonneg (fun _ _ => Finset.sum_nonneg (fun _ _ => Complex.normSq_nonneg _))
  have z1 : ∑ l, ∑ m, Complex.normSq (O1v q c A B l m) = 0 := by linarith
  have z2 : ∑ l, ∑ m, Complex.normSq (O2v q c A B l m) = 0 := by linarith
  intro l m
  have hz : ∀ (g : ι → ι → ℂ), ∑ l, ∑ m, Complex.normSq (g l m) = 0 → g l m = 0 := by
    intro g hg
    have hl := (Finset.sum_eq_zero_iff_of_nonneg (fun i _ => Finset.sum_nonneg
      (fun j _ => Complex.normSq_nonneg (g i j)))).mp hg l (Finset.mem_univ l)
    have hm := (Finset.sum_eq_zero_iff_of_nonneg (fun j _ =>
      Complex.normSq_nonneg (g l j))).mp hl m (Finset.mem_univ m)
    exact Complex.normSq_eq_zero.mp hm
  exact (hz _ z1, hz _ z2)

/-! ## Daleckii–Krein divided difference -/

/-- `F(p, p') = (p ln p − p' ln p')/(p − p')`, `= 1 + ln p` on the diagonal. -/
def Fdk (p p' : ℝ) : ℝ := if p = p' then 1 + log p else (p * log p - p' * log p') / (p - p')

lemma Lker_symm (p p' : ℝ) : Lker p p' = Lker p' p := by
  unfold Lker
  by_cases h : p = p'
  · subst h; rfl
  · have h' : p' ≠ p := fun e => h e.symm
    simp only [h, h', ↓reduceIte]
    rw [← neg_div_neg_eq]; ring_nf

lemma Fdk_symm (p p' : ℝ) : Fdk p p' = Fdk p' p := by
  unfold Fdk
  by_cases h : p = p'
  · subst h; rfl
  · have h' : p' ≠ p := fun e => h e.symm
    simp only [h, h', ↓reduceIte]
    rw [← neg_div_neg_eq]; ring_nf

/-- `ln p_r − F_lr = −p_l L_lr` and `ln p_l − F_lr = −p_r L_lr` (multiplied by `c`). -/
lemma F_rel {cl cm : ℝ} (hl : 0 < cl) (hm : 0 < cm) :
    cm * log (cm ^ 2) = cm * Fdk (cl ^ 2) (cm ^ 2) - cl ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cm ∧
    cl * log (cl ^ 2) = cl * Fdk (cl ^ 2) (cm ^ 2) - cm ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cl := by
  have hl2 : 0 < cl ^ 2 := by positivity
  have hm2 : 0 < cm ^ 2 := by positivity
  unfold Fdk Lker
  by_cases h : cl ^ 2 = cm ^ 2
  · simp only [h, ↓reduceIte]
    constructor <;> field_simp <;> ring
  · simp only [h, ↓reduceIte]
    have hd : cl ^ 2 - cm ^ 2 ≠ 0 := sub_ne_zero.mpr h
    constructor <;> field_simp <;> ring

/-- `F_rel` for `c ≥ 0`: with the conventions `0 ln 0 = 0` and the continuous values
`F_lr = ln p_l` if `p_r = 0 < p_l`, the relations persist on kernel entries. -/
lemma F_rel' {cl cm : ℝ} (hl : 0 ≤ cl) (hm : 0 ≤ cm) :
    cm * log (cm ^ 2) = cm * Fdk (cl ^ 2) (cm ^ 2) - cl ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cm ∧
    cl * log (cl ^ 2) = cl * Fdk (cl ^ 2) (cm ^ 2) - cm ^ 2 * Lker (cl ^ 2) (cm ^ 2) * cl := by
  rcases hl.eq_or_lt with h0 | hlp
  · subst h0
    rcases hm.eq_or_lt with h1 | hmp
    · subst h1; simp
    · have hm2 : (0:ℝ) ≠ cm ^ 2 := (by positivity : (0:ℝ) < cm ^ 2).ne
      unfold Fdk
      simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, hm2, ↓reduceIte,
        log_zero, zero_mul, zero_sub, mul_zero, sub_zero, zero_mul]
      have : cm ^ 2 ≠ 0 := fun e => hm2 e.symm
      refine ⟨?_, ?_⟩ <;> first | trivial | field_simp
  · rcases hm.eq_or_lt with h1 | hmp
    · subst h1
      have hl2 : cl ^ 2 ≠ 0 := (by positivity : (0:ℝ) < cl ^ 2).ne'
      unfold Fdk
      simp only [ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true, zero_pow, hl2, ↓reduceIte,
        log_zero, mul_zero, sub_zero, zero_mul]
      refine ⟨?_, ?_⟩ <;> first | trivial | field_simp
    · exact F_rel hlp hmp

/-! ## (S:Q12), first line -/

lemma Q1_term (q κ α β cl cm gl gm F L : ℝ) (A B : ℂ)
    (hgm : gm = cm * F - cl ^ 2 * L * cm) (hgl : gl = cl * F - cm ^ 2 * L * cl) :
    (conj (A * (cm : ℂ) - (q : ℂ) * (cl : ℂ) * conj B)
      * ((A * (gm : ℂ) - (q : ℂ) * (gl : ℂ) * conj B
          + ((q * κ - α : ℝ) : ℂ) * A * (cm : ℂ) + ((κ - q * β : ℝ) : ℂ) * (cl : ℂ) * conj B)
        - (F : ℂ) * (A * (cm : ℂ) - (q : ℂ) * (cl : ℂ) * conj B))).re
    = (q * κ - α) * (cm ^ 2 * Complex.normSq A)
      + (q ^ 2 * β - q * κ) * (cl ^ 2 * Complex.normSq B)
      + (κ * (1 - q ^ 2) + q * (α - β)) * (cl * cm * (A * B).re)
      - cl ^ 2 * cm ^ 2 * L * Complex.normSq A
      - q ^ 2 * (cl ^ 2 * cm ^ 2 * L * Complex.normSq B)
      + q * (cl * cm * (cl ^ 2 + cm ^ 2) * L * (A * B).re) := by
  subst hgm hgl
  simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.add_re,
    Complex.add_im, Complex.mul_re, Complex.mul_im, Complex.ofReal_re, Complex.ofReal_im,
    Complex.conj_re, Complex.conj_im]
  ring
```

```
167
168 /-- **Eq. (S:Q12), first line**, derived from the Euler equation and `[T, O₁]`. -/
169 theorem Q1_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
170     (α β κ q nA nB : ℝ)
171     (hA0 : nA = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A l r))
172     (hB1 : nB + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B l r)) :
173     (∑ l, ∑ m, conj (O1v q c A B l m)
174       * ((A l m * ((c m * log (c m ^ 2) : ℝ) : ℂ)
175           - (q : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B l m)
176           + ((q * κ - α : ℝ) : ℂ) * A l m * (c m : ℂ)
177           + ((κ - q * β : ℝ) : ℂ) * (c l : ℂ) * conj (B l m))
178         - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O1v q c A B l m)).re
179     = Q1 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) := by
180   unfold Q1
181   rw [hA0, hB1]
182   unfold kk sfA sfC
183   simp only [Complex.re_sum, Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib]
184   refine Finset.sum_congr rfl (fun l _ => Finset.sum_congr rfl (fun m _ => ?_))
185   obtain (h1, h2) := F_rel' (hc l) (hc m)
186   have hs : √(c l ^ 2 * c m ^ 2) = c l * c m := by
187     rw [show c l ^ 2 * c m ^ 2 = (c l * c m) * (c l * c m) by ring]
188     exact sqrt_mul_self (mul_nonneg (hc l) (hc m))
189   unfold O1v
190   rw [Q1_term q κ α β (c l) (c m) _ _ (Fdk (c l ^ 2) (c m ^ 2)) (Lker (c l ^ 2) (c m ^ 2))
191     (A l m) (B l m) h1 h2, hs]
192
193 /-! ## (S:Q12), second line -/
194
195 lemma Q2_term (q κ α β cx cy gx gy F L : ℝ) (A B : ℂ)
196     (hgx : gx = cx * F - cy ^ 2 * L * cx) (hgy : gy = cy * F - cx ^ 2 * L * cy) :
197     (conj ((q : ℂ) * (cx : ℂ) * conj A - (cy : ℂ) * B)
198       * (((q : ℂ) * (gx : ℂ) * conj A - (gy : ℂ) * B
199           + ((q * α - κ : ℝ) : ℂ) * (cx : ℂ) * conj A + ((β - q * κ : ℝ) : ℂ) * (cy : ℂ) * B)
200         - (F : ℂ) * ((q : ℂ) * (cx : ℂ) * conj A - (cy : ℂ) * B))).re
201     = (q ^ 2 * α - q * κ) * (cx ^ 2 * Complex.normSq A)
202       + (q * κ - β) * (cy ^ 2 * Complex.normSq B)
203       + (κ * (1 - q ^ 2) - q * (α - β)) * (cx * cy * (A * B).re)
204       - q ^ 2 * (cx ^ 2 * cy ^ 2 * L * Complex.normSq A)
205       - cx ^ 2 * cy ^ 2 * L * Complex.normSq B
206       + q * (cx * cy * (cx ^ 2 + cy ^ 2) * L * (A * B).re) := by
207   subst hgx hgy
208   simp only [Complex.normSq_apply, Complex.sub_re, Complex.sub_im, Complex.add_re,
209     Complex.add_im, Complex.mul_re, Complex.mul_im, Complex.ofReal_re, Complex.ofReal_im,
210     Complex.conj_re, Complex.conj_im]
211   ring
212
213 /-- **Eq. (S:Q12), second line**, derived from the Euler equation and `[T, O₂]`. -/
214 theorem Q2_from_euler (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : ι → ι → ℂ)
215     (α β κ q nA nB : ℝ)
216     (hA1 : nA + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A l r))
217     (hB0 : nB = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B l r)) :
218     (∑ l, ∑ m, conj (O2v q c A B l m)
219       * (((q : ℂ) * ((c m * log (c m ^ 2) : ℝ) : ℂ) * conj (A m l)
220           - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B m l
221           + ((q * α - κ : ℝ) : ℂ) * (c m : ℂ) * conj (A m l)
222           + ((β - q * κ : ℝ) : ℂ) * (c l : ℂ) * B m l)
223         - (Fdk (c l ^ 2) (c m ^ 2) : ℂ) * O2v q c A B l m)).re
224     = Q2 α β κ q nA nB (kk c A B) (sfA c A) (sfA c B) (sfC c A B) := by
225   rw [Finset.sum_comm]
226   unfold Q2
227   rw [hA1, hB0]
228   unfold kk sfA sfC
229   simp only [Complex.re_sum, Finset.mul_sum, ← Finset.sum_add_distrib, ← Finset.sum_sub_distrib]
230   refine Finset.sum_congr rfl (fun x _ => Finset.sum_congr rfl (fun y _ => ?_))
231   obtain (h1, h2) := F_rel' (hc y) (hc x)
232   have hs : √(c x ^ 2 * c y ^ 2) = c x * c y := by
233     rw [show c x ^ 2 * c y ^ 2 = (c x * c y) * (c x * c y) by ring]
234     exact sqrt_mul_self (mul_nonneg (hc x) (hc y))
235   rw [Lker_symm] at h1 h2
236   unfold O2v
237   rw [Q2_term q κ α β (c x) (c y) _ _ (Fdk (c y ^ 2) (c x ^ 2)) (Lker (c x ^ 2) (c y ^ 2))
238     (A x y) (B x y) h1 h2, hs]
239
240 /-! ## Rigidity at a stable stationary point -/
241
242 /-- **Rigidity.** For `m` pairs with the hypotheses of Lemma S5, Schmidt coefficients `c ≥ 0`
243 and the sum rules, if the second variations (S:Q12) along all test vectors `O₁,ⱼψ, O₂,ⱼψ` —
244 computed from the Euler equation — sum to a nonnegative number, then every test vector
245 vanishes: `(a_j − q_j b_j†)ψ = 0` and `(q_j a_j† − b_j)ψ = 0` for all `j`. -/
246 theorem stationary_rigidity {m : ℕ} (κ α β w : Fin m → ℝ)
247     (h : ∀ j, Hyp (κ j) (α j) (β j) (w j))
248     (c : ι → ℝ) (hc : ∀ l, 0 ≤ c l) (A B : Fin m → ι → ι → ℂ) (nA nB : Fin m → ℝ)
249     (hA0 : ∀ j, nA j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (A j l r))
250     (hA1 : ∀ j, nA j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (A j l r))
251     (hB0 : ∀ j, nB j = ∑ l, ∑ r, c r ^ 2 * Complex.normSq (B j l r))
252     (hB1 : ∀ j, nB j + 1 = ∑ l, ∑ r, c l ^ 2 * Complex.normSq (B j l r))
253     (hstab : 0 ≤ ∑ j,
254       ((∑ l, ∑ m', conj (O1v (exp (-w j)) c (A j) (B j) l m')
255         * ((A j l m' * ((c m' * log (c m' ^ 2) : ℝ) : ℂ)
256             - ((exp (-w j) : ℝ) : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B j l m')
257             + ((exp (-w j) * κ j - α j : ℝ) : ℂ) * A j l m' * (c m' : ℂ)
```

```
            + ((κ j - exp (-w j) * β j : ℝ) : ℂ) * (c l : ℂ) * conj (B j l m'))
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O1v (exp (-w j)) c (A j) (B j) l m')).re
      + (∑ l, ∑ m', conj (O2v (exp (-w j)) c (A j) (B j) l m')
        * ((((exp (-w j) : ℝ) : ℂ) * ((c m' * log (c m' ^ 2) : ℝ) : ℂ) * conj (A j m' l)
            - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B j m' l
            + ((exp (-w j) * α j - κ j : ℝ) : ℂ) * (c m' : ℂ) * conj (A j m' l)
            + ((β j - exp (-w j) * κ j : ℝ) : ℂ) * (c l : ℂ) * B j m' l)
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O2v (exp (-w j)) c (A j) (B j) l m')).re)) :
    ∀ j l m', O1v (exp (-w j)) c (A j) (B j) l m' = 0 ∧ O2v (exp (-w j)) c (A j) (B j) l m' = 0 := by
  have hQ : ∀ j, ((∑ l, ∑ m', conj (O1v (exp (-w j)) c (A j) (B j) l m')
        * ((A j l m' * ((c m' * log (c m' ^ 2) : ℝ) : ℂ)
            - ((exp (-w j) : ℝ) : ℂ) * ((c l * log (c l ^ 2) : ℝ) : ℂ) * conj (B j l m')
            + ((exp (-w j) * κ j - α j : ℝ) : ℂ) * A j l m' * (c m' : ℂ)
            + ((κ j - exp (-w j) * β j : ℝ) : ℂ) * (c l : ℂ) * conj (B j l m'))
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O1v (exp (-w j)) c (A j) (B j) l m')).re
      + (∑ l, ∑ m', conj (O2v (exp (-w j)) c (A j) (B j) l m')
        * ((((exp (-w j) : ℝ) : ℂ) * ((c m' * log (c m' ^ 2) : ℝ) : ℂ) * conj (A j m' l)
            - ((c l * log (c l ^ 2) : ℝ) : ℂ) * B j m' l
            + ((exp (-w j) * α j - κ j : ℝ) : ℂ) * (c m' : ℂ) * conj (A j m' l)
            + ((β j - exp (-w j) * κ j : ℝ) : ℂ) * (c l : ℂ) * B j m' l)
          - (Fdk (c l ^ 2) (c m' ^ 2) : ℂ) * O2v (exp (-w j)) c (A j) (B j) l m')).re)
      = Q1 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
          (sfA c (B j)) (sfC c (A j) (B j))
        + Q2 (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (kk c (A j) (B j)) (sfA c (A j))
          (sfA c (B j)) (sfC c (A j) (B j)) := by
    intro j
    rw [Q1_from_euler c hc (A j) (B j) (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (hA0 j) (hB1 j),
      Q2_from_euler c hc (A j) (B j) (α j) (β j) (κ j) (exp (-w j)) (nA j) (nB j) (hA1 j) (hB0 j)]
  simp only [hQ] at hstab
  -- γ₀ = min_j γ_j (or 1 if there are no pairs)
  obtain ⟨γ0, hγ0, hγ⟩ : ∃ γ0 : ℝ, 0 < γ0 ∧ ∀ j, γ0 ≤ gam (κ j) (α j) (β j) (w j) := by
    rcases Nat.eq_zero_or_pos m with hm | hm
    · subst hm; exact ⟨1, one_pos, fun j => j.elim0⟩
    · have : Nonempty (Fin m) := ⟨⟨0, hm⟩⟩
      obtain ⟨j0, -, hj0⟩ := Finset.exists_min_image Finset.univ
        (fun j => gam (κ j) (α j) (β j) (w j)) Finset.univ_nonempty
      exact ⟨_, gam_pos (h j0), fun j => hj0 j (Finset.mem_univ j)⟩
  have hz := (rigidity_multi κ α β w h γ0 hγ0 hγ c hc A B nA nB hA0 hA1 hB0 hB1).2 hstab
  intro j l m'
  exact defects_zero (exp (-w j)) c (A j) (B j) (hz j) l m'

end Thm2Cert
```